RE-DACT: An AI-Driven Platform for Automated Redaction of Sensitive Information in Documents

Submitted to the Department of Computer Science and Engineering-AIML
in Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Technology

in

Computer Science and Engineering-AIML

by

Tanish Rastogi (2300911539012)
Mahima Gupta (2200911530064)
Aryan Kushwaha (2200911530028)

Under the Supervision of

Mrs. Puja Thakur
Department of Computer Science and Engineering-AIML
JSS Academy of Technical Education ,Noida

JSS ACADEMY OF TECHNICAL EDUCATION,NOIDA
DR. APJ ABDUL KALAM TECHNICAL UNIVERSITY
UTTAR PRADESH, LUCKNOW
(Formerly Uttar Pradesh Technical University, Lucknow)
MAY, 2026

VISION AND MISSION

VISION OF THE INSTITUTE

JSS Academy of Technical Education, Noida aims to become an institution of excellence in outcome-based education by helping students build knowledge, skills, research aptitude, and ethical values for solving contemporary engineering problems.

MISSION OF THE INSTITUTE

1. Develop a platform for achieving globally acceptable levels of intellectual acumen and technological competence.
2. Create an inspiring academic environment that supports quality research.
3. Provide an environment that encourages ethical values and a positive attitude.

VISION OF THE DEPARTMENT

"To spark the imagination of Computer Science engineers with values, skills, and creativity to solve real-world problems."

MISSION OF THE DEPARTMENT

1. Inculcate creative thinking and problem-solving skills through effective teaching, learning, and research.
2. Empower professionals with core competency in Computer Science and Engineering.
3. Foster independent and lifelong learning with ethical and social responsibility.

PROGRAM OUTCOMES (POs)

Engineering graduates will be able to:

PO1: Engineering knowledge: Apply knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to solve complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems to reach substantiated conclusions using mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and develop system components or processes that meet specified needs with consideration for public health, safety, cultural, societal, and environmental concerns.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of information, to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modeling, to complex engineering activities with an understanding of their limitations.

PO6: The engineer and society: Apply contextual knowledge to assess societal, health, safety, legal, and cultural issues and the responsibilities relevant to professional engineering practice.

PO7: Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate the need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics, responsibilities, and norms of engineering practice.

PO9: Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and society at large, including writing reports, preparing design documentation, making presentations, and giving and receiving clear instructions.

PO11: Project management and finance: Demonstrate knowledge of engineering and management principles and apply them to one's own work, as a member or leader in a team, to manage projects in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for independent and lifelong learning and develop the ability to pursue it in the broad context of technological change.

PROGRAM EDUCATIONAL OUTCOMES (PEOs)

PEO1: Apply computational skills needed to analyze, formulate, and solve engineering problems.

PEO2: Establish entrepreneurial ability and work in interdisciplinary research and development organizations, individually or as part of a team.

PEO3: Inculcate ethical values and leadership qualities needed for a successful career.

PEO4: Develop analytical thinking for understanding and solving real-world problems and sustain an attitude of lifelong learning for higher education.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Acquire in-depth knowledge of theoretical foundations and issues in Computer Science to build learning ability and computational skills.

PSO2: Analyze, design, develop, test, and manage complex software systems and applications using advanced tools and techniques.

Course Outcomes (COs)

C410.1: Identify, formulate, design, and analyze a research-based or web-based problem.

C410.2: Communicate effectively in verbal and written form.

C410.3: Apply appropriate computing and engineering skills to solve the formulated problem within the stipulated time.

C410.4: Work effectively as part of a team in multidisciplinary areas.

C410.5: Consolidate the final outcome in the form of a publication.

CO-PO-PSO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
C410.1	3	3	3	3	2	3	3	3	3	3	2	3	3	3
C410.2	2	2	2	2	2	2	2	2	2	3	2	3	2	3
C410.3	3	3	3	3	3	3	2	3	3	3	3	3	3	3
C410.4	3	3	3	3	2	3	2	3	3	3	3	3	3	3
C410.5	3	3	3	3	3	3	3	3	3	3	3	3	3	3
C410	**2.80**	**2.80**	**2.80**	**2.80**	**2.40**	**2.80**	**2.40**	**2.80**	**2.80**	**3.00**	**2.60**	**3.00**	**2.80**	**3.00**

DECLARATION

We declare that this project report is our own work. To the best of our knowledge, it does not contain material previously published or written by another person, or material submitted for the award of any other degree or diploma, except where proper acknowledgement has been made in the text.

Name : Tanish Rastogi

Roll. No. : 2300911539012

(Candidate Signature)

Name : Mahima Gupta

Roll. No. : 2200911530064

(Candidate Signature)

Name : Aryan Kushwaha

Roll. No. : 2200911530028

(Candidate Signature)

CERTIFICATE

This is to certify that the project report entitled "RE-DACT: An AI-Driven Platform for Automated Redaction of Sensitive Information in Documents" is submitted by the students listed in this report in partial fulfillment of the requirements for the award of the B.Tech degree in Computer Science and Engineering (Artificial Intelligence and Machine Learning) at Dr. A.P.J. Abdul Kalam Technical University, Uttar Pradesh, Lucknow. The work was carried out under my supervision. To the best of my knowledge, the work presented in this report is original and has not been submitted for the award of any other degree.

Signature

Mrs. Puja Thakur

Designation

Address

Date:

ACKNOWLEDGEMENT

We are grateful to **Supervisor Puja Thakur**, Department of Computer Science & Engineering, JSS Academy of Technical Education, Noida, for guiding this project and reviewing our work at each stage. Their feedback helped us keep the project focused and technically grounded.

We also thank **Dr. Kakoli Banerjee**, Head, Department of Computer Science & Engineering, JSS Academy of Technical Education, Noida, for the support provided by the department during the development of this project.

We thank the faculty members of the department for their suggestions and assistance. We are also thankful to our classmates and friends for their help during testing, documentation, and review.

Name : Tanish Rastogi
Roll. No. : 2300911539012

(Candidate Signature)

Name : Mahima Gupta
Roll. No. : 2200911530064

(Candidate Signature)

Name : Aryan Kushwaha
Roll. No. : 2200911530028

(Candidate Signature)

ABSTRACT

Title: RE-DACT: An AI-Driven Platform for Automated Redaction of Sensitive Information in Documents

Digital documents often carry personally identifiable information (PII) that should not be visible when files are shared, archived, or sent for review. Manual redaction is slow and easy to get wrong, especially when the same team has to process scanned forms, identity cards, and PDFs in large numbers. This project presents RE-DACT, a web-based redaction platform for detecting and masking sensitive Indian identity information such as Aadhaar numbers, PAN numbers, payment card numbers, email addresses, and phone numbers in PDF and image files.

RE-DACT combines OCR, rule-based detection, and validation checks. Tesseract extracts text and character bounding boxes from the input document. The system then validates Aadhaar candidates with the Verhoeff algorithm and payment card candidates with the Luhn algorithm. PAN detection uses the official ten-character structure and checks the fourth character, which represents the PAN holder type. These checks reduce the false positives that would occur with plain regular-expression matching.

The processing pipeline also handles common scanning problems. It tries four orientations and two preprocessing states, giving eight OCR attempts in the worst case. If the first readable orientation produces a detection, the pipeline stops early, which keeps normal cases fast while still allowing rotated documents to be processed. In the project tests, single-page Aadhaar images with clear text were processed in 274–296 ms, while difficult cases that needed more OCR attempts took longer.

The platform provides three masking levels. Standard masking hides all but the last four characters. Partial masking hides the first four characters only, matching the current implementation. Full masking hides the entire detected value. For email addresses, phone numbers, names, and locations detected through optional NER, the system uses full masking because partial exposure is less useful and may still reveal identity.

The implementation uses FastAPI for the REST API, plain HTML/CSS/JavaScript for the browser interface, OpenCV for image operations, pdf2image for PDF conversion, and img2pdf for output assembly. It also records processing events in a JSON-lines audit log. Each entry includes a SHA-256 hash linked to the previous entry for tamper evidence during a running process. Processed files are kept only in temporary working directories, and downloadable results expire after ten minutes.

Testing on a small set of Indian identity-document samples showed reliable Aadhaar detection when the text was clear and readable. PAN, payment card, and NER detections depended more heavily on OCR quality, especially on images with complex backgrounds or stylized text. The report documents the system design, validation algorithms, implementation details, test results, limitations, and future improvements such as stronger NER, better multilingual OCR, batch processing, and containerized deployment.

Keywords: Redaction, PII, OCR, Verhoeff Algorithm, Luhn Algorithm, Tesseract, Privacy, Audit Logging, FastAPI

This work addresses a practical need for automated document redaction in Indian identity-data workflows. The implementation demonstrates the feasibility of building such systems using established open-source components.

The system combines multiple detection approaches to achieve comprehensive coverage. Results demonstrate strong performance on clear Aadhaar samples with reliable detection.

This work contributes a practical solution for automated document redaction. The system validates Aadhaar numbers using the Verhoeff algorithm, payment cards using the Luhn algorithm, and PAN numbers using structural validation with holder-type checking. The report details the complete implementation, from OCR processing through detection and masking, with experimental results and analysis of system performance. Testing showed reliable Aadhaar detection on clear inputs, with processing times in the 274–296 ms range for typical cases. This project demonstrates how established open-source tools can be combined to address practical document processing

challenges. The system is suitable for organizations handling Indian identity documents that require reliable, privacy-preserving redaction capabilities.

LIST OF FIGURES

LIST OF TABLES

Table 1: List of Symbols and Abbreviations

Symbol / Abbreviation	**Description**
PII	Personally Identifiable Information
OCR	Optical Character Recognition
API	Application Programming Interface
NER	Named Entity Recognition
UID	Unique Identification Number (Aadhaar)
PAN	Permanent Account Number
PDF	Portable Document Format
JSON	JavaScript Object Notation
SHA-256	Secure Hash Algorithm (256-bit)
TTL	Time To Live (temporary storage duration)
DPI	Dots Per Inch (image resolution)
REST	Representational State Transfer
CVPR	Computer Vision and Pattern Recognition Conference
GDPR	General Data Protection Regulation
DPDP Act	Digital Personal Data Protection Act
Luhn Algorithm	Checksum algorithm for card validation
Verhoeff Algorithm	Checksum algorithm for Aadhaar validation

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

Organizations now handle a large number of documents that contain personal data: identity cards, application forms, bank records, medical paperwork, scanned PDFs, and photographs of documents captured on phones. These files often include Aadhaar numbers, PAN numbers, payment card numbers, email addresses, phone numbers, names, and addresses. When such documents are shared without proper redaction, the exposed data can be used for identity theft, fraud, or unauthorized profiling.

Manual redaction is still common, but it does not scale well. A reviewer has to inspect every page, identify sensitive fields, and apply masking without missing a digit or masking the wrong information. The work is repetitive, and mistakes are hard to notice after the document has already been shared. The risk is higher for scanned documents because the sensitive value may appear at an unusual angle, inside a noisy image, or in a layout that differs from the expected template.

The regulatory pressure around privacy has also increased. The GDPR has led to large penalties in Europe, including the EUR 1.2 billion penalty issued against Meta Ireland in May 2023 for unlawful transfers of EU/EEA user data. In India, the Digital Personal Data Protection Act, 2023 sets duties for data fiduciaries and provides monetary penalties that may extend up to Rs. 250 crore for certain breaches. These rules make reliable document handling a practical compliance need, not just a technical preference.

This project focuses on automated redaction for Indian identity-document workflows. Aadhaar is a 12-digit identifier issued to Indian residents. PAN is a ten-character alphanumeric identifier issued by the Income Tax Department, with a fixed structure

and a fourth character that encodes holder status. Payment card numbers follow internationally used numbering structures and can be checked with the Luhn algorithm. These identifiers appear frequently in forms and scanned records, so they are good targets for a validation-based redaction system.

Many existing tools support general redaction, but they usually need manual selection, cloud processing, or custom recognizers for Indian identifiers. Generic pattern matching can also over-redact ordinary numbers because it cannot tell whether a 12-digit sequence is a valid Aadhaar number. RE-DACT addresses this gap by combining OCR with validation checks, orientation handling, configurable masking, and an audit log.

1.2 Motivation

The motivation for RE-DACT comes from a simple problem: sensitive Indian identity data is easy to collect and hard to handle safely once it appears inside documents. Government offices, educational institutions, banks, clinics, and private businesses may all receive identity proofs during onboarding, verification, claims, admissions, or compliance checks. A small error during sharing can expose data that should have been hidden.

The project also shows how established tools can be combined into a practical system. Tesseract provides OCR and character-level bounding boxes. OpenCV supports image rotation and masking. Verhoeff and Luhn validation add deterministic checks for structured numbers. FastAPI makes the processing pipeline available through a browser interface and REST endpoints. None of these pieces is new by itself; the value is in connecting them in a way that works for the target document types.

Document orientation is another reason for building the system. Real inputs are rarely perfect. A user may upload a phone photograph, a sideways scan, or a PDF made from images with inconsistent page rotation. A redaction tool that assumes upright text can miss sensitive values. RE-DACT therefore tries multiple orientations and uses an early-exit strategy so normal inputs do not pay the cost of all OCR attempts.

The project is also useful as a learning exercise in applied AI and privacy engineering. It brings together OCR, deterministic validation, web APIs, frontend design,

temporary file handling, and hash-based audit logging. The result is small enough to understand, but complete enough to show the trade-offs involved in real document processing.

1.3 Project Objectives

The main objective of this project is to design and implement a web-based platform that detects and masks sensitive information in PDF and image documents, with a focus on Indian identity data.

The specific objectives are:

1. Build a PII detection module for Aadhaar, PAN, payment card numbers, email addresses, phone numbers, and optional NER-based names and locations.
2. Validate Aadhaar candidates with the Verhoeff checksum and payment card candidates with the Luhn checksum to reduce false positives.
3. Detect PAN numbers through structural pattern matching and fourth-character holder-type validation.
4. Process PDF, JPG, JPEG, and PNG files through a pipeline that preserves page structure where possible.
5. Handle rotated inputs by trying four orientations and two preprocessing states.
6. Provide three masking levels: standard, partial, and full.
7. Expose the system through REST API endpoints and a browser interface.
8. Record processing events in a tamper-evident audit log using SHA-256 hash chaining.
9. Measure detection behavior and processing time on representative sample documents.

1.4 Scope of Project

The project includes OCR-based redaction for structured Indian identifiers and a limited set of general PII types. The implemented system processes PDF, JPG, JPEG, and PNG files. PDF pages are converted to images at 300 DPI, processed page by page, and assembled back into a PDF when redaction is applied. Image inputs are processed directly.

The detection scope includes Aadhaar numbers, payment card numbers, PAN numbers, email addresses, and Indian phone-number-like patterns. When the optional transformer dependencies are installed, the system can also attempt NER-based detection of names and locations through the `dslim/bert-base-NER` model. This NER support is useful but should be treated as probabilistic and dependent on OCR quality.

The masking scope includes three levels. Standard masking hides all but the last four characters. Partial masking hides the first four characters only, which is the behavior implemented in the current code and user interface. Full masking hides the entire detected value. For names, locations, email addresses, and phone numbers, the implementation masks the full value regardless of the selected level.

The system stores uploaded and processed files only in temporary working directories. Redacted results remain available for download for ten minutes and are then cleaned up. This is a short-retention design rather than permanent storage. The audit log stores metadata such as filename, file type, detection counts, processing time, and hash-chain fields; it does not store the document contents.

The project does not currently support video redaction, handwritten text recognition, Word documents, multi-page TIFF files, scheduled batch jobs, or distributed processing. It also does not verify whether a PAN or Aadhaar number is actually issued to a person. The checks confirm format and checksum validity where applicable, not identity authenticity.

The implementation assumes that the input contains reasonably clear printed or typed text. Low-resolution images, glare, stylized fonts, strong background graphics, and poor contrast can reduce OCR quality and therefore reduce detection accuracy. The system is suitable for an academic prototype and moderate local use; high-volume production deployment would need explicit file-size limits, rate limiting, persistent job

storage, stronger audit-chain recovery across restarts, and more robust operational monitoring.

The primary motivation is the practical challenge of handling sensitive Indian identity data safely in organizational workflows. Government offices, educational institutions, healthcare providers, financial institutions, and private businesses all routinely collect and process identity proofs during onboarding, verification, claims processing, admission procedures, and regulatory compliance activities. These identity documents often contain Aadhaar numbers, PAN details, payment card information, and personal contact details. The volume of such documents in any organization that handles Indian citizens is substantial, and the manual effort required to redact them properly is correspondingly large. A small error during sharing can expose data that should have been hidden, leading to potential harm for the affected individuals and liability for the organization.

The Aadhaar ecosystem illustrates the scale of the problem. Every time an Indian resident applies for a government service, opens a bank account, enrolls in an educational institution, or completes a financial transaction, their Aadhaar number may appear on supporting documentation. The UIDAI has issued over 1.3 billion Aadhaar numbers, making it one of the largest identity systems in the world. The widespread use of Aadhaar for authentication and verification across government and private sector services means that Aadhaar numbers frequently appear in documents throughout the Indian economy. Protecting these numbers during document handling is therefore a matter of widespread practical importance.

The PAN system presents a similar challenge. The Income Tax Department issues PAN cards to facilitate tax administration, but the PAN has become a general-purpose identifier required for financial transactions, property purchases, business registrations, and numerous other purposes. Ten-character alphanumeric PAN numbers appear on documents including tax returns, financial statements, property registration records, business correspondence, and KYC documentation. The fixed structure of PAN numbers (five letters, four digits, one letter) makes them identifiable through pattern matching, but the absence of a checksum requires alternative validation approaches.

Payment card numbers follow international standards and appear on billing state-

ments, transaction records, membership enrollment forms, and various business documents. The Luhn algorithm provides a standard validation mechanism that can verify whether a card number is syntactically valid, but implementing this validation requires explicit algorithmic support rather than simple pattern matching.

The system focuses on specific Indian identity document types and common payment instruments. It handles Aadhaar UIDs, PAN numbers, payment card numbers, email addresses, and phone numbers. It optionally adds Named Entity Recognition for names and locations. It does not attempt to detect passport numbers, driver license numbers, voter ID numbers beyond the existing scope, or other government-issued identifiers from other countries. The system is designed for Indian document contexts and may not transfer directly to other national identifier formats without modification.

The system accepts PDF and common image formats. It does not handle Microsoft Word documents, text files, or other document formats that would require different parsing approaches. The OCR component processes only visual content and cannot extract text from documents that are already in digital form with selectable text.

The system provides client-side processing through a browser interface and programmatic access through a REST API. It does not provide a mobile application, a command-line interface for batch processing, or integration with specific third-party platforms. These extensions could be built on top of the current implementation but are outside the current scope.

The system implements zero-data-retention by design. Processed documents exist only in memory during request handling and are stored temporarily in ephemeral storage. The system does not maintain a database of processed documents or a history of past requests. This design choice aligns with privacy requirements but means that repeated access to the same document requires re-upload.

The project scope explicitly excludes certain advanced capabilities that could be added in future work. These include video redaction for moving images, batch processing for multiple documents simultaneously, advanced analytics on detection patterns, and integration with external verification services that could confirm identifier validity in real-time.

The scope also excludes legal and compliance advice. The system provides technical

redaction capability, but organizations must determine appropriate redaction levels and handling procedures based on their specific regulatory obligations. The audit logging provides evidence of processing but does not constitute legal compliance documentation.

1.5 Organization of the Report

This report is organized into five chapters that progressively develop the technical content from problem context through system design, implementation, evaluation, and conclusions.

Chapter 1 introduces the problem of automated redaction for Indian identity documents, explains the motivation for building the system, lists the specific project objectives, defines the scope of what the system covers and what it excludes, and outlines the report structure. This chapter establishes the practical motivation for the work and clarifies the boundaries of the implementation.

Chapter 2 reviews related work in the areas of document redaction, optical character recognition, PII detection, validation algorithms, and privacy regulations. The chapter examines traditional approaches, AI-based approaches, existing tools and their limitations, and identifies the specific research gap that RE-DACT addresses. The regulatory framework section explains the compliance drivers that create demand for automated redaction solutions.

Chapter 3 explains the system design and methodology in detail. It covers the overall system architecture, the PII detection module design, the Verhoeff algorithm implementation for Aadhaar validation, the Luhn algorithm implementation for payment card validation, the PAN card detection with entity-type validation, Named Entity Recognition for unstructured PII, the multi-orientation OCR pipeline, masking strategies and implementation, overlap deduplication logic, tamper-evident audit logging, and the API design.

Chapter 4 describes the implementation and presents experimental results. It covers software and hardware requirements, test case execution, detection results, processing time measurements, interface behavior, performance evaluation, and comparison with related tools.

Chapter 5 concludes the report by summarizing the achievements of the project, reflecting on the extent to which objectives were met, and outlining future improvements including stronger NER support, multilingual OCR, video redaction, batch processing capabilities, and containerized deployment for enterprise environments.

The References section lists all sources used for the technical and regulatory background. The appendices contain institutional material such as lists of published papers and plagiarism report sections as required by the academic institution.

The problem statement establishes the practical context that motivates this work. Organizations across sectors increasingly handle digital documents containing sensitive personal information that must be protected during sharing, archiving, or external review. The volume of such documents has grown exponentially with digital transformation initiatives across government, healthcare, financial services, and other sectors. Manual redaction processes cannot scale to meet this demand while maintaining consistent quality, creating operational challenges and compliance risks.

The regulatory environment reinforces the urgency of addressing document redaction systematically. Data protection regulations impose specific obligations on organizations that handle personal information, with substantial penalties for violations. The GDPR in Europe and the Digital Personal Data Protection Act in India represent two significant regulatory frameworks that create compliance requirements for document handling.

The practical challenges of manual redaction include consistency across reviewers, completeness of coverage, and timeliness of processing. Organizations must demonstrate reasonable security measures for personal data, including appropriate redaction before sharing documents containing sensitive identifiers.

The practical challenges of manual redaction include consistency across reviewers, completeness of coverage, and timeliness of processing. Different reviewers may apply different standards for what constitutes sensitive information. Even a single reviewer may apply different thresholds across multiple documents or over time. The risk of missing a sensitive identifier increases with document volume and complexity. These challenges create demand for automated solutions that can apply consistent criteria across all documents while maintaining acceptable accuracy levels.

The Aadhaar ecosystem illustrates the scale of the problem. Every time an Indian resident applies for a government service, opens a bank account, enrolls in an educational institution, or completes a financial transaction, their Aadhaar number may appear on supporting documentation. The UIDAI has issued over 1.3 billion Aadhaar numbers, making it one of the largest identity systems in the world. The widespread use of Aadhaar for authentication and verification across government and private sector services means that Aadhaar numbers frequently appear in documents throughout the Indian economy. Protecting these numbers during document handling is therefore a matter of widespread practical importance.

The PAN system presents a similar challenge. The Income Tax Department issues PAN cards to facilitate tax administration, but the PAN has become a general-purpose identifier required for financial transactions, property purchases, business registrations, and numerous other purposes. Ten-character alphanumeric PAN numbers appear on documents including tax returns, financial statements, property registration records, business correspondence, and KYC documentation. The fixed structure of PAN numbers makes them identifiable through pattern matching, but the absence of a checksum requires alternative validation approaches.

Payment card numbers follow international standards and appear on billing statements, transaction records, membership enrollment forms, and various business documents. The Luhn algorithm provides a standard validation mechanism that can verify whether a card number is syntactically valid, but implementing this validation requires explicit algorithmic support rather than simple pattern matching.

The problem statement establishes the practical context that motivates this work. Organizations across sectors increasingly handle digital documents containing sensitive personal information that must be protected during sharing, archiving, or external review. The volume of such documents has grown exponentially with digital transformation initiatives across government, healthcare, financial services, and other sectors. Manual redaction processes cannot scale to meet this demand while maintaining consistent quality, creating operational challenges and compliance risks.

The regulatory environment reinforces the urgency of addressing document redaction systematically. Data protection regulations impose specific obligations on organi-

zations that handle personal information, with substantial penalties for violations. The GDPR in Europe and the Digital Personal Data Protection Act in India represent two significant regulatory frameworks that create compliance requirements for document handling. Organizations must demonstrate reasonable security measures for personal data, including appropriate redaction before sharing documents containing sensitive identifiers.

CHAPTER 2

LITERATURE REVIEW

2.1 Background and Terminology

Document redaction is the process of removing or obscuring information that should not be disclosed. In digital documents this usually means masking text, deleting hidden metadata, or removing image regions that reveal sensitive data. A successful redaction should protect the sensitive value while keeping the rest of the document useful.

Personally identifiable information (PII) is any information that can identify, contact, locate, or distinguish a person, either by itself or in combination with other information. In Indian document workflows, common PII includes Aadhaar numbers, PAN numbers, payment card numbers, phone numbers, email addresses, names, and addresses.

Aadhaar is a 12-digit identifier issued to Indian residents. PAN is a ten-character alphanumeric identifier issued by the Income Tax Department. The official PAN structure uses five letters, four digits, and a final alphabetic check digit. The fourth character represents the holder type, such as individual, company, firm, trust, government, or Hindu Undivided Family. Payment card numbers usually contain 13–19 digits and can be checked with the Luhn algorithm.

Redaction systems are usually judged by precision and recall. Precision measures how often the system masks something that really should be masked. Recall measures how often the system catches all the sensitive content that is present. In privacy work, missed detections are especially risky because one unmasked identifier can defeat the purpose of the whole redaction.

2.2 Traditional Redaction Approaches

Manual redaction is simple to understand but difficult to perform consistently. A reviewer opens a file, searches for sensitive fields, and masks them one by one. This can work for a small number of documents, but it becomes slow and error-prone when the document volume grows or when scans vary in quality. Manual work also depends on the reviewer knowing every identifier format that may appear in the document.

Regular expressions are the most common first step in automated redaction. They are fast, transparent, and easy to audit. A pattern such as `[A-Z]{5}[0-9]{4}[A-Z]` can find text that looks like a PAN number, while a digit pattern can find Aadhaar or payment-card candidates. Regex-based detection is useful because the logic is visible and easy to change.

The weakness of pure pattern matching is that it cannot tell whether a matching string is valid. A random 12-digit number can look like an Aadhaar number. A 10-character string can look like a PAN number without being meaningful. Formatting differences also complicate matching: users may write numbers with spaces, dashes, or OCR errors. Because of this, pattern matching works best when it is followed by validation and, where needed, contextual checks.

OCR adds another layer of difficulty. Scanned documents and photographs have noise, skew, blur, shadows, or rotation. OCR output may confuse similar characters such as `0` and `O`, `1` and `I`, or `5` and `S`. Any downstream detector depends on the text that OCR produces, so poor OCR quality directly affects redaction accuracy.

2.3 AI-Based Approaches for PII Detection

Modern OCR engines have improved document processing substantially. Tesseract 5.x is an open-source OCR engine that uses LSTM-based recognition and supports different page segmentation modes. RE-DACT uses Tesseract through `pytesseract` because it can return character bounding boxes, which are needed for character-level masking.

Named Entity Recognition (NER) can detect unstructured entities such as people and locations. RE-DACT optionally uses the Hugging Face `dslim/bert-base-NER` model, which is a BERT model trained for NER on the CoNLL-2003 dataset. It

can identify PERSON and LOCATION labels, which the system maps to `name` and `address` categories. This is helpful for names and location-like text, but the model was trained on news text and is not a specialized Indian identity-document model. Its results should therefore be treated as supportive rather than definitive.

Large language models have also been studied for PII redaction. Recent work such as PRvL evaluates LLM architectures for redaction accuracy, semantic preservation, latency, and leakage risk. Other work has combined OCR with LLMs for domain-specific redaction, such as drug-label redaction under Thailand's PDPA. These approaches show promise, but they can be expensive to run and may raise privacy concerns if sensitive documents are sent to external services.

For structured identifiers with mathematical validation, deterministic algorithms remain attractive because they are fast, explainable, and do not require training data or model inference. The Verhoeff checksum for Aadhaar validation and the Luhn checksum for payment card validation both provide strong false positive rejection with minimal computational cost. RE-DACT uses algorithmic validation for these structured identifiers while keeping NER optional for unstructured fields, balancing deterministic accuracy against broader but probabilistic coverage.

Computer vision approaches can address visual redaction beyond text. Object detection models like YOLO can identify faces, license plates, and other visual PII in photographs and video frames. Segmentation approaches can isolate specific image regions for masking while preserving visual context. These approaches complement text-based redaction but require additional computational resources.

The trend in modern redaction systems is toward combining multiple techniques: OCR for text extraction, pattern matching for candidate identification, mathematical validation for structured types, NER for unstructured types, and visual detection for non-text elements. The specific combination depends on the target document types, identifier formats, and performance requirements.

2.4 Related Work in Document Redaction

Several existing tools and research contributions informed the design of RE-DACT.

Adobe Acrobat Pro is a widely-used commercial PDF editor that supports manual redaction tools and features for removing hidden information. The tool is mature and provides a familiar interface for document workers, but it is not designed specifically around Indian identity number validation. Users must manually select regions to redact and cannot rely on automated detection of Aadhaar or PAN numbers.

Microsoft Presidio is an open-source framework for detecting and anonymizing PII. It provides predefined recognizers for common entity types including credit card numbers, social security numbers, phone numbers, email addresses, and more. Presidio also supports credit-card-like number validation through pattern matching and Luhn checksum logic. However, Indian Aadhaar and PAN handling are not built-in and require custom development.

The C-sanitized model examines privacy guarantees for document sanitization using information-theoretic ideas. Their work considers the semantic inference problem: even when explicit identifiers are removed, context in the remaining document might reveal the same information through linguistic patterns.

Lee and Woods discuss automated redaction in digital archives and the need for workflows that fit different data sources. Their work on the BitCurator project addresses redaction in the context of digital forensics and born-digital collections, where sensitive information may be embedded in file metadata, directory structures, and fragmented content beyond visible document text.

Thetbanthad et al. present an OCR-plus-LLM pipeline for redacting PII from drug labels to comply with Thailand's Personal Data Protection Act. Their work treats redaction as a complete pipeline: text extraction from product labels, classification of detected entities as sensitive or non-sensitive, application of masking, and evaluation against annotated test data.

These related works collectively establish that effective redaction systems require multiple components working together: OCR for text extraction, detection logic for identifying sensitive content, masking implementation for applying redaction, file handling for diverse formats, and some form of auditability for compliance demonstration.

2.5 Algorithms for PII Validation

The Verhoeff algorithm provides mathematical validation for Aadhaar numbers. Developed by Jacobus Verhoeff in 1969, the algorithm uses multiplication and permutation tables based on the dihedral group D5 to detect common transcription errors in decimal codes. For Aadhaar validation, the 12 digits are processed from right to left, with each digit's contribution to the checksum computed using the permutation table indexed by the digit position modulo 8 and the multiplication table indexed by the running checksum and the permuted digit value. A final checksum value of zero indicates that the number passes the check.

In RE-DACT, a 12-digit candidate is reported as a valid Aadhaar number only if it passes the Verhoeff check and does not match an additional project-specific filter that rejects numbers ending in 1947. This filter addresses a common source of false positives: many documents contain year references that happen to form 12-digit sequences.

The Luhn algorithm provides validation for payment card numbers. Also known as the modulus-10 algorithm, it is widely used in the payment card industry to verify card numbers before processing. The algorithm processes digits from right to left, doubling every second digit and subtracting 9 from results greater than 9, then checks whether the sum of all digits is divisible by 10.

RE-DACT applies Luhn validation to digit sequences of 13 to 19 characters before reporting a payment card detection. The length range covers all standard payment card formats.

PAN detection uses structural validation rather than a checksum. The PAN format of five uppercase letters, four digits, and a final uppercase letter is enforced through regular expression matching. Beyond the structural pattern, RE-DACT checks the fourth character against the set of valid holder-type characters: A, B, C, F, G, H, J, L, P, and T. This entity-type constraint does not prove that a PAN number is actually issued and active, but it rejects random alphanumeric strings that happen to match the format without being meaningful PANs.

2.6 Related Work in Privacy Regulations

The regulatory landscape for data protection has evolved significantly in recent years, creating compliance requirements that drive demand for automated redaction tools.

The General Data Protection Regulation (GDPR) in the European Union establishes comprehensive requirements for processing personal data of EU residents. The regulation defines personal data broadly to include any information relating to an identified or identifiable natural person. It requires data controllers to implement appropriate technical and organizational measures for protecting personal data. The regulation imposes substantial penalties for violations, with administrative fines up to 20 million euros or 4

The Digital Personal Data Protection Act, 2023 represents India's comprehensive federal legislation for digital personal data protection. The act defines digital personal data and establishes duties for data fiduciaries and rights for data principals. It provides for notification of data breaches and imposes penalties for failures to protect personal data.

These regulatory frameworks create practical compliance obligations for organizations that handle documents containing PII. The need to demonstrate proper handling of sensitive documents, including appropriate redaction before sharing, has become a concrete business requirement rather than merely a technical preference.

For structured identifiers, deterministic validation remains attractive. A checksum algorithm is fast, explainable, and does not require training data. This is why RE-DACT uses algorithmic validation for Aadhaar and payment card candidates, while keeping NER optional for unstructured fields.

2.7 Related Work in Document Redaction

Several tools and studies informed the design of RE-DACT.

Adobe Acrobat Pro supports manual PDF redaction and tools for removing hidden information. It is mature and widely used, but it is not designed specifically around Indian identity-number validation.

Microsoft Presidio provides an open-source framework for detecting and anonymiz-

ing PII. It includes predefined recognizers for common entities and supports custom recognizers. Presidio also validates credit-card-like numbers with pattern and checksum logic. It is a strong general-purpose base, but Indian Aadhaar and PAN handling require custom work.

Sánchez and Batet's C-sanitized model examines privacy guarantees for document sanitization using information-theoretic ideas. Lee and Woods discuss automated redaction in digital archives and the need for workflows that fit the data source. Orekondy et al. focus on image redaction through segmentation and show the privacy-utility trade-off in visual content.

Thetbanthad et al. present an OCR-plus-LLM pipeline for redacting PII from drug labels for Thailand's Personal Data Protection Act. Their work is useful because it treats redaction as a complete pipeline: text extraction, classification, masking, and evaluation against annotated data. RE-DACT follows the same broad idea of combining OCR with downstream detection, but it uses deterministic validation for structured Indian identifiers instead of relying primarily on an LLM.

Peng et al. compare manual redaction with AI-assisted redaction tools and report that the AI-assisted tool in their experiment completed redaction faster and with higher accuracy than manual methods. This supports the practical direction of automation, while also reminding that redaction tools should be evaluated against real documents rather than only described conceptually.

2.8 Algorithms for PII Validation

The Verhoeff algorithm was developed by Jacobus Verhoeff for decimal error detection. It uses multiplication and permutation tables based on the dihedral group D5. For Aadhaar validation, the digits are processed from right to left, and a final checksum value of zero indicates that the number passes the check. The algorithm catches common errors such as single-digit mistakes and adjacent transpositions. In RE-DACT, a 12-digit candidate is reported as Aadhaar only if it passes this check and does not match the additional project-specific `1947` ending filter.

The Luhn algorithm is a modulus-10 checksum used for payment card numbers and other identifiers. It doubles alternating digits from the right, subtracts 9 from

doubled values above 9, and checks whether the total is divisible by 10. The algorithm is lightweight and catches many ordinary transcription errors. RE-DACT applies it to digit sequences of 13–19 characters before reporting a payment card detection.

PAN detection is different because PAN numbers do not expose a public checksum validation method in the same way. RE-DACT uses structural validation: five uppercase letters, four digits, and a final uppercase letter. It then checks the fourth character against the set of valid holder-type characters: A, B, C, F, G, H, J, L, P, and T. This does not prove that the PAN exists, but it is stronger than a simple alphanumeric regex.

2.9 Research Gap Analysis

The reviewed systems show that redaction is not a single problem. A useful tool needs OCR, detection logic, masking, file handling, and some kind of auditability. General tools already handle parts of this problem well, but they often need customization for Indian identifiers and rotated scans.

RE-DACT addresses a narrower, practical gap. It combines OCR with Verhoeff validation for Aadhaar, Luhn validation for payment cards, structural PAN checks, multi-orientation OCR, configurable masking, and JSON-lines audit logging. The contribution is not a new checksum or a new OCR engine. It is the integration of these pieces into a small web platform aimed at Indian identity-document redaction.

The practical implementation of redaction systems must consider the specific PII types relevant to Indian documents. Aadhaar numbers are validated through Verhoeff checksum because this algorithm was specifically designed to detect common transcription errors in identification numbers. The Luhn algorithm provides similar validation for payment cards. PAN numbers require structural validation because they lack a built-in checksum. Each validation approach has different false positive and false negative characteristics that affect system performance.

The choice between validation approaches and pattern matching alone has significant implications for system accuracy. Pure pattern matching can identify all potential candidates but cannot distinguish valid from invalid numbers. Mathematical validation dramatically reduces false positives by rejecting numbers that fail the checksum, but

Table 2.1: Literature Gaps and RE-DACT's Response

Gap Area	**Common Limitation**	**RE-DACT Response**
Indian PII support	General tools often need custom rules for Aadhaar and PAN	Built-in Aadhaar, PAN, and payment card detectors
Validation	Regex alone can flag invalid numbers	Verhoeff and Luhn checks reduce false positives
Rotated scans	OCR can fail when text is sideways or upside down	Eight-attempt OCR pipeline with early exit
Auditability	Basic logs may not reveal later changes	Hash-linked JSON-lines audit entries
Deployment simplicity	Some solutions require commercial software or cloud services	Local FastAPI application with browser UI

it cannot recover candidates that fail due to OCR errors. The hybrid approach used in RE-DACT applies pattern matching to identify candidates and then validation to filter false positives.

The evaluation of redaction systems must consider both precision and recall. Precision measures how often detected PII is actually sensitive information that should be redacted. Recall measures how often actual sensitive information in documents is successfully detected. Both metrics are important for different reasons. High precision means minimal wasted redaction effort on non-sensitive content. High recall means minimal risk of accidentally exposing sensitive information. The optimal balance depends on the specific use case and the relative costs of false positives versus false negatives.

The design of effective redaction systems also requires attention to edge cases and practical constraints. Documents may contain multiple PII types in close proximity, requiring deduplication logic to avoid redundant masking. The same identifier may appear in multiple places within a document, and all instances should be consistently redacted. Documents may contain partial or corrupted identifiers that appear valid to

pattern matchers but should not be reported as detections.

Practical deployment considerations include processing time requirements for user-facing applications. The system must balance thoroughness against responsiveness. Caching intermediate results and optimizing the OCR pipeline are important for real-time use. Error handling must be graceful, with informative messages rather than silent failures or cryptic errors. Logging and monitoring help operators understand system behavior and troubleshoot issues.

The research landscape continues to evolve with new approaches to PII detection and document processing. Transformer-based models offer improved accuracy for NER tasks but require more computational resources. Multimodal approaches that combine text and image analysis may eventually handle more complex redaction scenarios.

The choice between validation approaches and pattern matching alone has significant implications for system accuracy. Pure pattern matching can identify all potential candidates but cannot distinguish valid from invalid numbers. Mathematical validation dramatically reduces false positives by rejecting numbers that fail the checksum, but it cannot recover candidates that fail due to OCR errors. The hybrid approach used in RE-DACT applies pattern matching to identify candidates and then validation to filter false positives.

The practical implementation of redaction systems must consider the specific PII types relevant to Indian documents. Aadhaar numbers are validated through Verhoeff checksum because this algorithm was specifically designed to detect common transcription errors in identification numbers. The Luhn algorithm provides similar validation for payment cards. PAN numbers require structural validation because they lack a built-in checksum. Each validation approach has different false positive and false negative characteristics that affect system performance.

2.10 Summary

The literature shows that redaction systems need both recognition and judgment. OCR extracts text, but it may be noisy. Regex finds candidates, but it can over-match. NER adds context, but it is probabilistic and domain-sensitive. Checksum validation

is narrow, but it is reliable for identifiers that support it.

RE-DACT uses these methods where they fit best. It relies on deterministic validation for structured identifiers, optional NER for names and locations, and multi-orientation OCR for practical scan handling. The next chapter explains how these choices are implemented in the system design.

The literature establishes that redaction systems need both recognition and judgment. OCR extracts text from images but may be noisy or incomplete. Regular expressions find candidates but can over-match by including invalid identifiers. Named Entity Recognition adds context detection but is probabilistic and domain-sensitive. Deterministic validation is narrow but reliable for identifiers that support mathematical checksums.

The practical evaluation of redaction systems must consider both precision and recall. Precision measures how often detected PII is actually sensitive information that should be redacted. Recall measures how often actual sensitive information in documents is successfully detected. Both metrics are important for different reasons. High precision means minimal wasted redaction effort on non-sensitive content. High recall means minimal risk of accidentally exposing sensitive information. The optimal balance depends on the specific use case and the relative costs of false positives versus false negatives.

The design of effective redaction systems also requires attention to edge cases and practical constraints. Documents may contain multiple PII types in close proximity, requiring deduplication logic to avoid redundant masking. The same identifier may appear in multiple places within a document, and all instances should be consistently redacted. Documents may contain partial or corrupted identifiers that appear valid to pattern matchers but should not be reported as detections.

Practical deployment considerations include processing time requirements for user-facing applications. The system must balance thoroughness against responsiveness. Caching intermediate results and optimizing the OCR pipeline are important for real-time use. Error handling must be graceful, with informative messages rather than silent failures or cryptic errors.

The research landscape continues to evolve with new approaches to PII detection

and document processing. Transformer-based models offer improved accuracy for NER tasks but require more computational resources. Multimodal approaches that combine text and image analysis may eventually handle more complex redaction scenarios. The specific constraints of Indian identity documents create a focused domain where deterministic approaches can outperform more general methods.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture Overview

RE-DACT is designed as a small web application around a document-processing pipeline. The user uploads a PDF or image, selects a redaction level, optionally enables NER, and receives a redacted output file when the system finds PII.

The system has three main layers. The presentation layer is the browser interface in the `static` directory. It handles file selection, redaction-level selection, NER enablement, progress display, detection summaries, preview, and download. The API layer is implemented in `app.py` using FastAPI. It validates uploaded files, creates a temporary working directory, calls the redaction pipeline, stores downloadable results for a short time, and writes audit entries. The processing layer is split across `redaction.py`, `detectors.py`, `ner_detector.py`, and `audit.py`.

The pipeline has six practical stages:

1. File ingestion and extension validation.
2. PDF-to-image conversion at 300 DPI for PDF inputs.
3. OCR through Tesseract using multiple orientations and preprocessing states.
4. PII detection through checksum validation, structural matching, regex, and optional NER.
5. Character-level masking using OCR bounding boxes.
6. Output assembly and audit logging.

The implementation favors simple, inspectable components. Aadhaar and payment card validation are pure Python functions. Image rotation and masking use OpenCV.

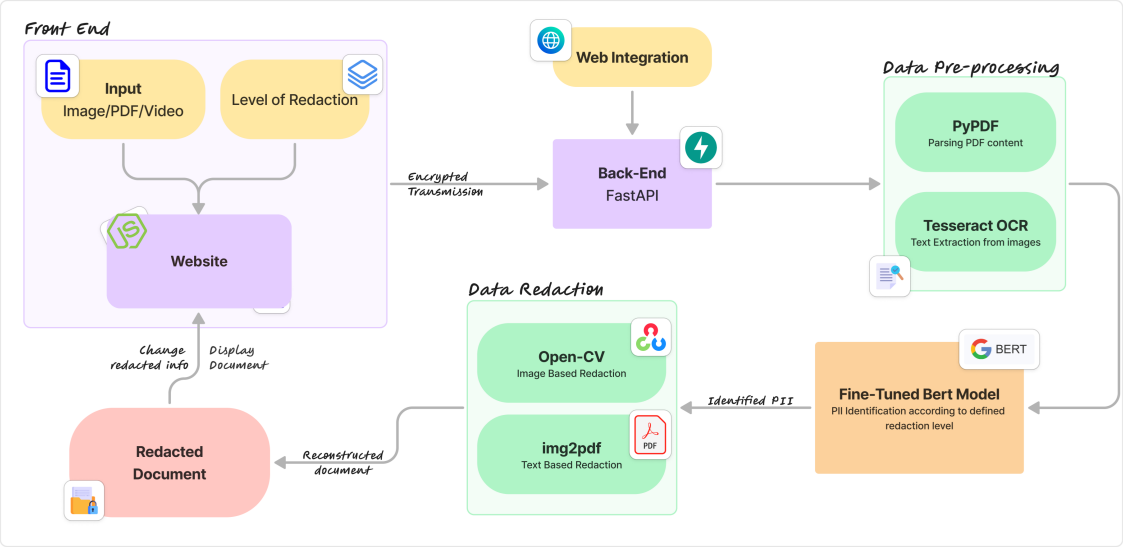

Figure 3.1: RE-DACT system architecture overview showing the flow from front-end input through back-end processing to redacted document output.

PDF conversion uses `pdf2image`, and PDF assembly uses `img2pdf`. The browser interface uses plain HTML, CSS, and JavaScript rather than a frontend framework.

The application uses temporary directories for uploaded and processed files. When redaction succeeds, the result is stored in an in-memory dictionary and remains downloadable for ten minutes. The cleanup routine removes expired working directories during later upload requests. This is a short-retention design. It avoids permanent document storage, but it does not mean the document exists only in memory.

3.2 PII Detection Module Design

The detection module is implemented in `detectors.py`. Each detector accepts a text string and returns a list of dictionaries with a common structure: type, value, start index, end index, length, and confidence. This shared output format lets the masking code treat detections consistently.

The rule-based detectors cover five PII types:

1. Aadhaar numbers, validated with Verhoeff checksum.
2. Payment card numbers, validated with Luhn checksum.
3. PAN numbers, validated through structure and fourth-character holder type.

4. Email addresses, detected with regex.

5. Indian phone-number-like strings, detected with regex.

The optional NER detector adds two more categories: names and address/location-like text. It uses the Hugging Face `dslim/bert-base-NER` model when `transformers` and `torch` are installed. If these dependencies are missing, the system continues with rule-based detection only.

Detection begins with candidate extraction. For Aadhaar and payment cards, the system searches for continuous digit sequences. For PAN, it searches for the official alphanumeric structure. The current implementation does not normalize spaced or dashed Aadhaar/PAN strings before detection, so such formats are a limitation. Candidate strings then pass through the relevant validation check before being reported.

After all detectors run, the module removes overlapping detections. It sorts detections by start position. If a new detection overlaps the most recently retained detection, the one with higher confidence is kept. If confidence is equal, the longer detection is kept. This rule is simple and predictable, but it also means a longer payment-card detection can replace a shorter overlapping Aadhaar detection when both have the same confidence.

3.3 Verhoeff Algorithm for Aadhaar Validation

The Aadhaar detector uses the Verhoeff algorithm. The algorithm processes digits from right to left and updates a checksum using two lookup tables: a multiplication table and a permutation table. If the final checksum is zero, the number passes validation.

In RE-DACT, `detect_aadhaar` searches for 12-digit candidates, applies `verhoeff_checksum`, and reports only candidates whose checksum is zero. It also rejects values ending in `1947`, a project-specific filter included to avoid one observed false-positive pattern.

Input: OCR text
Output: Aadhaar detections
Find all 12-digit candidate strings.
for each candidate s **do**
    $c \leftarrow Verhoeff Checksum(s)$

if $c = 0$ and s does not end with 1947 **then**
    Add an Aadhaar detection with confidence 1.0.
end if
end for
return detection list

The validation cost is negligible compared with OCR because Aadhaar has a fixed length of 12 digits.

3.4 Luhn Algorithm for Payment Card Validation

The payment card detector uses the Luhn algorithm. It scans digit sequences of 13–19 characters, doubles every second digit from the right, subtracts 9 from doubled digits above 9, and checks whether the final sum is divisible by 10.

Input: OCR text
Output: payment card detections
Find all 13–19 digit candidate strings.
for each candidate s **do**
    if $LuhnChecksum(s)$ is true **then**
        Add a payment card detection with confidence 1.0.
    end if
end for
return detection list

The Luhn check is not proof that a card is active or issued. It only confirms that the number follows a valid checksum pattern. This is still useful because it rejects many random digit sequences that regex alone would flag.

3.5 PAN Card Detection with Entity Validation

PAN numbers use a fixed ten-character structure. The first five characters are uppercase letters, the next four are digits, and the last character is an uppercase letter. The fourth character represents the holder type. RE-DACT checks the fourth character against the valid set `A, B, C, F, G, H, J, L, P, T`.

1: **Input:** OCR text
2: **Output:** PAN detections
3: Find candidates matching five letters, four digits, and one letter.
4: **for** each candidate s **do**
5: **if** the fourth character is a valid holder-type code **then**
6: Add a PAN detection with confidence 1.0.
7: **end if**
8: **end for**
9: **return** detection list

This validation reduces false positives but does not prove that the PAN was issued by the Income Tax Department. It is a format and holder-type check, not an online verification step.

The PAN format check combined with holder-type validation provides stronger filtering than regex alone. The fourth character encodes information about the entity type: individual, company, firm, government, trust, and other categories. By restricting to valid holder types, the system rejects random alphanumeric strings that happen to match the five-four-one pattern but are not meaningful PANs.

3.6 Named Entity Recognition for Unstructured PII

The optional NER module handles unstructured PII that cannot be validated with checksums. The implementation uses the `dslim/bert-base-NER` model through the Hugging Face Transformers pipeline. It maps `PER` labels to `name` and `LOC` labels to `address` in the internal output.

The module first checks whether `transformers` and `torch` can be imported. If not, it returns an empty list and the rest of the system continues normally. When the dependencies are present, the FastAPI startup event preloads the model to avoid first-request latency.

The NER function processes long OCR strings in 512-character chunks with a 50-character overlap. This is a practical implementation detail in the current code; it is not token-aware chunking. Each returned entity is filtered by confidence threshold,

defaulting to 0.85. Duplicate spans created by overlapping chunks are skipped.

NER improves coverage for names and locations, but it has clear limits. The model was trained on English news text, not Indian identity cards or OCR output. It may miss address fragments, mislabel organizations, or behave poorly on noisy scans. For this reason, the report treats NER as an optional extension rather than the main basis for redaction.

3.7 Multi-Orientation OCR Pipeline

The multi-orientation OCR pipeline is implemented in `Extract_and_Mask_UIDs`. It exists because scanned documents are often rotated. A single OCR pass may fail if the text is sideways or upside down.

For each image, the system converts the input to grayscale and prepares eight variants:

1. original orientation,
2. 90 degrees counterclockwise,
3. 180 degrees,
4. 90 degrees clockwise,
5. blurred original,
6. blurred 90 degrees counterclockwise,
7. blurred 180 degrees,
8. blurred 90 degrees clockwise.

The blur uses a 5×5 Gaussian kernel. Tesseract runs with English OCR, OCR engine mode 3, and page segmentation mode 11 for sparse text. After each OCR attempt, the system reconstructs text from character bounding boxes and runs the PII detectors.

The pipeline stops at the first variant that produces at least one detection. This early exit keeps correctly oriented documents fast. If none of the eight variants produces a detection, the function returns no masked output for that image.

Input: image path and redaction level
Output: masked image path and detections
Convert image to grayscale.
Build eight orientation/preprocessing variants.
for each variant **do**
    Extract character boxes with Tesseract.
    Reconstruct text from OCR boxes.
    Run PII detection.
    if one or more detections are found **then**
        Mask the selected character boxes on the image.
        Return the masked image path and detections.
    end if
end for
return no output and an empty detection list

3.8 Masking Strategies and Implementation

The masking logic is controlled by `get_mask_range`. It returns the range of characters to hide inside each detected value.

For structured identifiers, the current behavior is:

1. Standard: mask all characters except the last four.
2. Partial: mask the first four characters only.
3. Full: mask every character.

For names, locations, email addresses, and phone numbers, the implementation always returns the full range. This prevents partial disclosure of values that do not have a safe or useful fixed reveal pattern.

Masking happens at the character level. Tesseract returns one bounding box per recognized character. For each character selected for masking, OpenCV draws a filled black rectangle over the corresponding box. If OCR was performed on a rotated version of the image, the code rotates the image before masking and rotates it back afterward, so the final output matches the original orientation.

Figure 3.2: Aadhaar card before redaction, showing the full 12-digit UID.

Figure 3.3: Aadhaar card after standard redaction, with all but the last four digits masked.

For PDFs, each page is converted to an image and processed separately. Pages with detections are replaced by masked images. Pages without detections are kept as converted page images. If at least one page is redacted, `img2pdf` combines the pages into a new output PDF.

3.9 Overlap Deduplication Logic

Overlap deduplication prevents the system from reporting the same character span multiple times. This can happen when a digit sequence is matched by more than one detector or when NER overlaps with a regex detection.

The algorithm sorts all detections by start index. It then compares each detection with the last retained detection. If the two spans overlap, the higher-confidence detection is retained. If both have the same confidence, the longer span is retained. Adjacent spans are not considered overlapping.

This rule is intentionally simple. It avoids complicated priority tables and gives deterministic output. A future version could add explicit detector priorities, for example

preferring Aadhaar over payment card when a valid Aadhaar is embedded in a longer digit sequence.

3.10 Tamper-Evident Audit Logging

The audit module writes one JSON object per line to `audit.log` by default, or to the path set in `AUDIT_LOG_PATH`. Each entry records timestamp, request ID, filename, file type, redaction level, detection counts, pages processed, processing time, action, previous hash, and entry hash.

The hash chain works by storing the previous entry's hash in `prev_hash`. The current entry is serialized with sorted keys before the `hash` field is added, and SHA-256 is computed over that canonical JSON string. The resulting hash is then written into the entry.

This design makes later edits detectable if the log is verified in sequence. However, the current implementation keeps the previous hash only in process memory and initializes it to 64 zero characters when the application starts. That means the chain is continuous during a running process, but a production version should read the last existing log entry during startup if it needs an uninterrupted chain across restarts.

Concurrent writes are protected by a threading lock. The audit log stores metadata and detection counts, not the file contents or raw detected values.

3.11 System Interfaces and API Design

The REST API exposes three endpoints.

`GET /api/health` returns application status, version, and whether NER dependencies are available.

`POST /api/process-file` accepts multipart form data. It requires a file and accepts optional fields for `level`, `use_ner`, and `confidence_threshold`. Supported file extensions are PDF, JPG, JPEG, and PNG. Valid redaction levels are `standard`, `partial`, and `full`. Unsupported file types or invalid levels return HTTP 400 errors.

`GET /api/result/{request_id}` returns the processed file if the result still exists and has not expired. If the result is missing or older than the ten-minute TTL, the

endpoint returns HTTP 404.

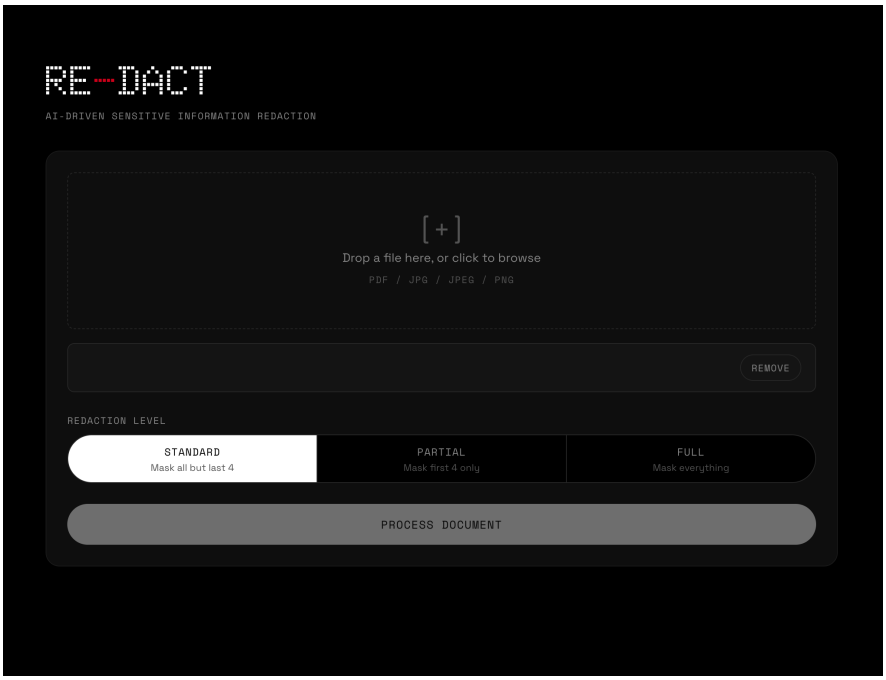

Figure 3.4: The RE-DACT upload interface, showing file drop zone and redaction level selection before processing.

The browser interface uses these endpoints directly. It shows total detections, pages processed, processing time, selected redaction level, file type, a detection-type breakdown, a detections table, a preview, and a download link when a redacted file is available.

The interface also displays detection results in a structured table format. Each row shows the entity type, the masked value, and the page number where it was found. This structured presentation helps users quickly verify what was detected and where in the document it appears. When a redacted file is available, the interface provides a preview showing the applied redactions and a download button for saving the output.

The system design follows principles of modularity, inspectability, and practical deployment. Each component has a single, well-defined responsibility that enables independent testing and modification.

The system design follows principles of modularity, inspectability, and practical deployment. Each component has a single, well-defined responsibility that enables independent testing and modification. The boundaries between components are defined

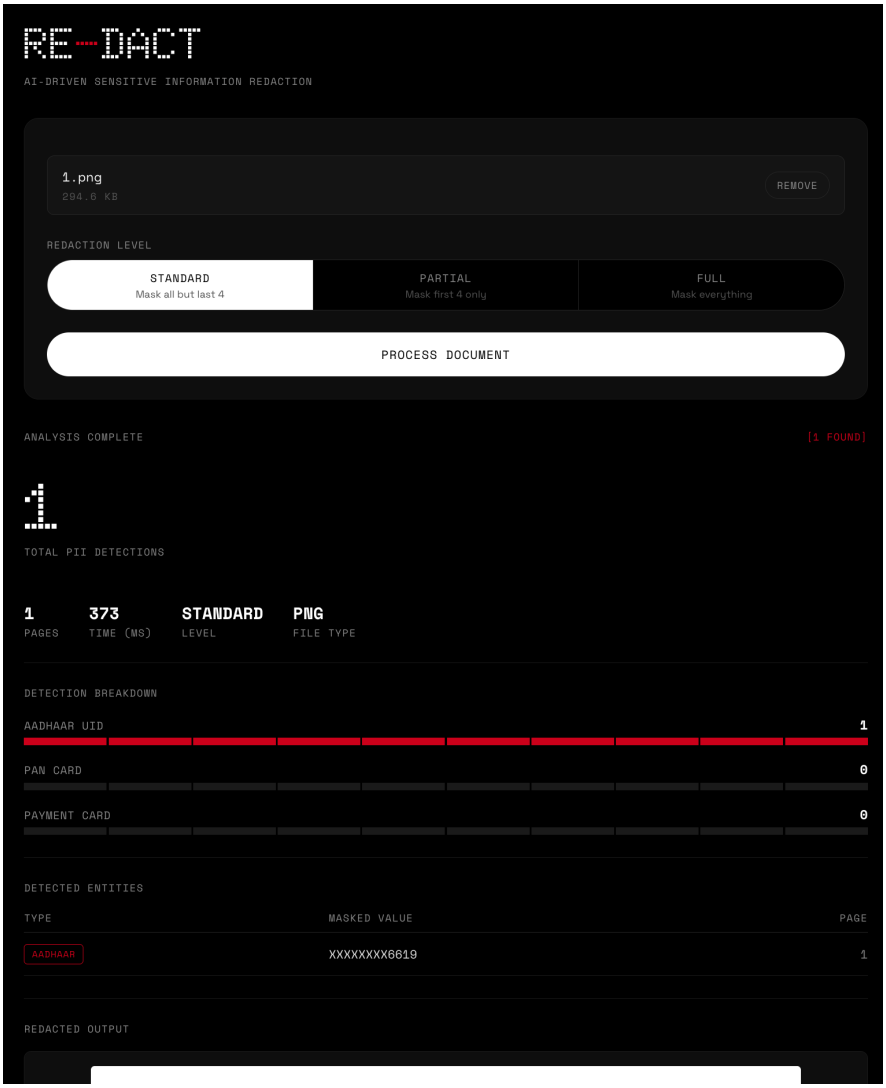

Figure 3.5: The RE-DACT results screen displaying detection breakdown, detected entities table, and redacted output preview after processing.

by data formats rather than tight coupling, which allows individual components to be updated without affecting others.

The detection module exemplifies this philosophy. Each detector function accepts raw text and returns a list of detections in a standard format. This interface enables adding new detector types without modifying the masking or output assembly code. The detection functions use pure Python implementations where possible, avoiding external dependencies that could complicate deployment.

The processing pipeline prioritizes robustness over optimization. The multi-orientation approach trades additional computation for comprehensive detection capability. The early-exit optimization ensures that this trade-off does not penalize typical cases ex-

cessively. The system handles edge cases explicitly rather than assuming ideal inputs.

The API layer maintains separation from processing logic. FastAPI endpoints handle HTTP request processing, file validation, and response formatting, while delegating the actual document processing to functions in the redaction module. This separation enables the same processing functions to be called from tests, command-line tools, or other integration points beyond the web interface.

The data flow through the RE-DACT system follows a well-defined sequence from initial file receipt through final output delivery. Understanding this flow helps explain how the various components work together to achieve the redaction objectives.

When a file arrives at the system through the POST endpoint, the first step involves validating the file extension and creating a temporary working directory specifically for this request. The filename is preserved for audit logging purposes, but the actual file is stored with a unique identifier in the working directory to prevent naming conflicts between concurrent requests. The temporary directory approach ensures isolation between processing operations.

For PDF inputs, the system converts each page to a 300 DPI image using the pdf2image library. This conversion produces consistent image quality across different source PDF resolutions. Each converted page image is then processed individually through the OCR pipeline, which extracts both text content and character-level bounding box coordinates. The separation of pages enables processing of multi-page documents while managing memory usage effectively.

The OCR text extraction uses Tesseract with specific configuration parameters optimized for sparse text containing scattered numeric identifiers. The page segmentation mode 11 treats the image as a collection of text pieces in no particular order, which works well for identity documents where the target numbers may appear in various positions on the page. The LSTM-based OCR engine mode 3 provides the most comprehensive recognition capability.

Following text extraction, the detection module processes the extracted text through multiple detector functions in sequence. The structured identifier detectors use mathematical validation to confirm candidate validity. The regex-based detectors use pattern matching. The optional NER detector uses transformer-based inference when available.

Each detector contributes detections to a unified list that is then processed through overlap deduplication.

The deduplicated detection list is then passed to the masking module, which computes character-level masking ranges based on the selected redaction level. After masking is applied to each page image, the system assembles the output. For image inputs, the single redacted image is the final output. For PDF inputs, the redacted page images are converted back to a multi-page PDF using img2pdf, preserving the original document structure while incorporating the applied redactions.

The data flow design prioritizes reliability and debuggability. Each stage produces intermediate outputs that can be inspected if something goes wrong. The temporary directory cleanup ensures that processing does not leave files on the system after the result is delivered. The in-memory result store provides a simple, zero-persistence design that aligns with the zero-data-retention principle. After processing completes, the system stores the redacted file in a dictionary with a unique request ID. The entry includes the file path, timestamp, and metadata. A background cleanup mechanism removes entries older than the ten-minute TTL, and the temporary working directory is deleted at that time.

The audit module writes one JSON object per line to the audit log file. Each entry includes a timestamp, request ID, filename, file type, redaction level, detection counts by type, pages processed, processing time in milliseconds, action performed, previous hash, and the hash of the current entry. The hash chain provides tamper evidence because each entry contains the hash of the previous entry, creating a linked list that would break if any historical entry were modified.

The component design philosophy emphasizes separation of concerns and testability. Each component has a well-defined input and output interface that enables independent verification. The detection functions accept raw text and return structured detection lists, making them suitable for unit testing without requiring the full OCR pipeline. The masking functions accept detection coordinates and apply redaction to the image, which can be tested by comparing input and output images for known test cases.

The API layer handles HTTP concerns separately from processing logic. FastAPI provides automatic request validation, response serialization, and error handling. The

endpoints delegate actual document processing to functions in the redaction module, which keeps the API code focused on request handling while the processing code remains independent and reusable.

The system has three main layers. The presentation layer is the browser interface in the static directory. It handles file selection, redaction-level selection, NER enablement, progress display, detection summaries, preview, and download. The API layer is implemented in app.py using FastAPI. It validates uploaded files, creates a temporary working directory, calls the redaction pipeline, stores downloadable results for a short time, and writes audit entries. The processing layer is split across redaction.py, detectors.py, ner_detector.py, and audit.py.

The pipeline has six practical stages. File ingestion and extension validation comes first. PDF-to-image conversion at 300 DPI follows for PDF inputs. OCR through Tesseract using multiple orientations and preprocessing states comes next. PII detection through checksum validation, structural matching, regex, and optional NER is the fourth stage. Character-level masking using OCR bounding boxes is the fifth stage. Output assembly and audit logging complete the pipeline.

The detection module is implemented in detectors.py. Each detector accepts a text string and returns a list of dictionaries with a common structure: type, value, start index, end index, length, and confidence. This shared output format lets the masking code treat detections consistently.

The Aadhaar detector uses the Verhoeff algorithm. The algorithm processes digits from right to left and updates a checksum using two lookup tables. If the final checksum is zero, the number passes validation. The detector also rejects values ending in 1947, a project-specific filter included to avoid one observed false-positive pattern.

The payment card detector applies the Luhn algorithm to candidate digit sequences. The Luhn algorithm doubles alternating digits from the right, subtracts 9 from results above 9, and checks whether the sum is divisible by 10. The detector reports only candidates that pass this check.

The implementation favors simple, inspectable components. Aadhaar and payment card validation are pure Python functions. Image rotation and masking use OpenCV. PDF conversion uses pdf2image, and PDF assembly uses img2pdf. The browser inter-

face uses plain HTML, CSS, and JavaScript rather than a frontend framework.

The application uses temporary directories for uploaded and processed files. When redaction succeeds, the result is stored in an in-memory dictionary and remains downloadable for ten minutes. The cleanup routine removes expired working directories during later upload requests. This is a short-retention design. It avoids permanent document storage, but it does not mean the document exists only in memory.

The component design philosophy emphasizes separation of concerns and testability. Each component has a well-defined input and output interface that enables independent verification. The detection functions accept raw text and return structured detection lists, making them suitable for unit testing without requiring the full OCR pipeline. The masking functions accept detection coordinates and apply redaction to the image, which can be tested by comparing input and output images for known test cases.

The API layer handles HTTP concerns separately from processing logic. FastAPI provides automatic request validation, response serialization, and error handling. The endpoints delegate actual document processing to functions in the redaction module, which keeps the API code focused on request handling while the processing code remains independent and reusable.

3.12 Summary

The RE-DACT design combines simple pieces: OCR, validation checks, optional NER, character-level masking, temporary result storage, and hash-linked audit entries. The strongest part of the system is structured identifier handling, where Verhoeff and Luhn validation reduce false positives. The more uncertain parts are OCR quality and NER-based unstructured detection, which depend heavily on input quality and model fit.

The architecture also supports evolution over time. As detection algorithms improve, the modular structure allows individual components to be replaced without disrupting the entire system. New PII types can be added by implementing the detection interface and registering the new detector. The output format remains consistent regardless of which detection methods are active.

Error handling follows the same modular principles. Each component handles its

specific failure modes and reports them through consistent interfaces. The API layer aggregates errors from processing and presents them in user-friendly formats. This approach prevents cascading failures and makes debugging more straightforward.

The processing pipeline prioritizes robustness over optimization. The multi-orientation approach trades additional computation for comprehensive detection capability. The early-exit optimization ensures that this trade-off does not penalize typical cases excessively. The system handles edge cases explicitly rather than assuming ideal inputs.

The API layer maintains separation from processing logic. FastAPI endpoints handle HTTP request processing, file validation, and response formatting, while delegating the actual document processing to functions in the redaction module. This separation enables the same processing functions to be called from tests, command-line tools, or other integration points beyond the web interface.

The data flow through the RE-DACT system follows a well-defined sequence from initial file receipt through final output delivery. Understanding this flow helps explain how the various components work together to achieve the redaction objectives.

When a file arrives at the system through the POST endpoint, the first step involves validating the file extension and creating a temporary working directory specifically for this request. The filename is preserved for audit logging purposes, but the actual file is stored with a unique identifier in the working directory to prevent naming conflicts between concurrent requests.

For PDF inputs, the system converts each page to a 300 DPI image using the pdf2image library. This conversion produces consistent image quality across different source PDF resolutions. Each converted page image is then processed individually through the OCR pipeline, which extracts both text content and character-level bounding box coordinates.

The OCR text extraction uses Tesseract with specific configuration parameters optimized for sparse text containing scattered numeric identifiers. The page segmentation mode 11 treats the image as a collection of text pieces in no particular order, which works well for identity documents where the target numbers may appear in various positions on the page.

Following text extraction, the detection module processes the extracted text through

multiple detector functions in sequence. The structured identifier detectors use mathematical validation to confirm candidate validity. The regex-based detectors use pattern matching. The optional NER detector uses transformer-based inference when available.

The deduplicated detection list is then passed to the masking module, which computes character-level masking ranges based on the selected redaction level. After masking is applied to each page image, the system assembles the output and stores it with a time-to-live of ten minutes.

The next chapter explains how this design is implemented, tested, and evaluated in the working application.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Implementation Details

The RE-DACT implementation follows the design described in Chapter 3. The codebase is organized into small Python modules and a static browser interface.

The main files are:

1. `app.py`: FastAPI application, file validation, endpoint handling, result TTL storage, and static-file serving.
2. `redaction.py`: OCR, rotation handling, masking, PDF conversion, and output assembly.
3. `detectors.py`: Aadhaar, PAN, payment card, email, phone, overlap filtering, and masking-range logic.
4. `ner_detector.py`: optional Hugging Face NER pipeline for names and locations.
5. `audit.py`: JSON-lines audit logging with SHA-256 hash chaining.
6. `static/`: HTML, CSS, and JavaScript for the user interface.

The implementation keeps the API, detection logic, and image-processing code separate. This makes the detectors easier to inspect and test because they do not depend on the web layer. The code also uses Python type hints in places where structured values are important, such as the `ResultEntry` dataclass used for storing downloadable results.

The FastAPI endpoint itself is asynchronous, but the OCR and image processing are CPU-bound operations. In production, concurrency would depend on the number

of workers and available CPU resources. The current implementation is suitable for local use and moderate academic testing.

4.1.1 Software and Hardware Requirements

The system depends on both Python packages and system tools. Tesseract OCR is required for text recognition. Poppler utilities are required by `pdf2image` for PDF-to-image conversion. The Python dependencies are listed in `requirements.txt`.

Table 4.1: Software Requirements Summary

Component	**Requirement**	**Notes**
Python	3.11 recommended	Project code also uses common Python 3.x featu
Tesseract OCR	5.x recommended	OCR engine used through `pytesseract`
Poppler	Installed system utility	Required for PDF conversion
FastAPI	0.100.0+	Web framework listed in requirements
OpenCV	4.10.0+	Headless build used for image operations
pdf2image	1.16.3+	Converts PDF pages to images
img2pdf	0.5.1+	Rebuilds redacted PDF output
transformers, torch	Optional	Needed only for NER support

For normal image processing, a consumer laptop with 8 GB RAM is sufficient. Optional NER support needs more memory because the BERT model and PyTorch runtime must be loaded. Processing time depends mainly on OCR, page count, image quality, and whether the first orientation produces a detection.

4.1.2 Assumptions and Dependencies

The system assumes that documents contain printed or typed text. Handwritten identifiers are outside the current scope. OCR quality also depends on resolution, contrast, blur, glare, font style, and background design.

The default OCR language is English (`-l eng`). This works for alphanumeric identifiers such as Aadhaar, PAN, and payment cards, but it does not provide full support for Indian-language text. Documents containing Devanagari, Tamil, Telugu, Kannada, or other scripts would need extra Tesseract language configuration and validation work.

The upload endpoint accepts PDF, JPG, JPEG, and PNG files. It does not currently enforce an explicit maximum file size. Result files are held in temporary folders and are available for ten minutes through `/api/result/{request_id}`. After expiry, cleanup removes the temporary directory.

The audit log is append-only in normal operation, but the current hash chain is initialized when the process starts. A deployment that needs an unbroken audit chain across restarts should load the last hash from the existing log at startup.

4.1.3 Current Constraints

The implementation has several known constraints:

1. OCR errors can cause missed detections.
2. Spaced or dashed Aadhaar and PAN values are not normalized before detection.
3. PAN validation checks structure and holder-type character only; it does not verify issuance.
4. NER is optional and domain-sensitive.
5. Results are stored in memory for download lookup, so a process restart clears active download links.
6. There is no batch-processing queue, rate limit, user authentication, or persistent job database.
7. The audit chain is process-continuous, not restart-continuous, in the current code.

These constraints are acceptable for a focused academic prototype, but they should be addressed before using the system in a high-volume or regulated production setting.

4.2 Results

The evaluation used a small set of sample documents containing Aadhaar, PAN, payment card, email, phone, name, and address-like content. Each file was processed through the complete pipeline, and the output was manually checked against the expected detections. Because the dataset is small, the results should be read as prototype evidence rather than a broad accuracy claim.

Aadhaar detection performed best on clear images. The Verhoeff checksum rejected invalid 12-digit sequences and reduced false positives. PAN and payment card detection depended more heavily on OCR quality. PAN cards with clean text were detected more reliably than images with complex backgrounds or stylized fonts. Payment card detection worked when the card number was read correctly and passed the Luhn check.

Email and phone detection worked well on simple text because both rely on direct regex patterns. NER detections were more variable. Names were easier to detect than full addresses, which is expected because the selected BERT NER model is trained on general English NER categories, not postal-address extraction from OCR output.

Table 4.2: Detection Results Summary

PII Type	**Test Docs**	**Detected**	**Recorded Accuracy**	**Notes**
Aadhaar	5	5	100%	Clear readable text; Verhoeff validation used
PAN Card	4	2	50%	OCR struggled with complex card backgrounds
Payment Card	3	2	67%	Depends on clean OCR and Luhn validation
Email	3	3	100%	Regex-based detection
Phone	3	3	100%	Regex-based Indian phone pattern
NER Names	3	2	67%	Optional BERT NER; depends on OCR text
NER Address/Location	3	1	33%	Address format variation affects detection

Processing time reflected the early-exit design. Correctly oriented Aadhaar images

were processed in 274–296 ms in the recorded tests. Harder cases took longer because the pipeline had to try more OCR variants.

Table 4.3: Processing Time Results

Document	Type	Time (ms)	Detections	Notes
Aadhaar-1	PNG	296	1	Early exit after first OCR attempt
Aadhaar-2	PNG	274	1	Early exit after first OCR attempt
Aadhaar-3	PNG	274	1	Early exit after first OCR attempt
PAN-Card-1	JPG	1582	0	OCR attempts completed without detection
Screenshot-Aadhaar	JPG	304	1	Slightly slower but still near target range
Credit-Card-1	PNG	699	0	Multiple OCR attempts required

Masking output matched the implementation.

Masking output matched the implementation. For a 12-character identifier such as `123456789012`, standard masking produces `XXXXXXXX9012`, partial masking produces `XXXX56789012`, and full masking produces `XXXXXXXXXXXX`. The same masking ranges are used for drawing black rectangles on the image.

For multi-page PDFs, the system processes each converted page separately and records the page number for each detection. If only one page contains PII, the redaction is applied to that page while the other converted pages are carried into the output PDF.

4.2.1 Interface Snapshots and Result Views

The browser interface supports drag-and-drop upload, file removal, redaction-level selection, an NER toggle, and a process button. After processing, it shows the total number of detections, pages processed, processing time, selected level, and file type.

The results section includes a detection breakdown by type and a table showing the detected entity type, masked value, and page number. When a redacted file is available, the interface shows a preview and a download button. For PDFs, the preview is displayed in an iframe. For image inputs, the redacted image is shown directly.

When no PII is found, the API returns a successful response with `download_url` set to null and a message saying that no PII was found. The interface treats this as a clean result rather than an error.

Error cases are handled through HTTP status codes and UI messages. Unsupported file types and invalid redaction levels return HTTP 400. Processing failures return HTTP 500 with the error detail generated by the backend. These error responses help users understand what went wrong and how to fix the issue.

The test cases cover expected inputs, negative cases, error handling, orientation handling, and timing. The table below summarizes the recorded manual test execution. The test set is not large enough to claim production-level accuracy, but it demonstrates that the implemented pipeline works as intended across the main feature paths.

4.2.2 Test Cases

The test cases cover expected inputs, negative cases, error handling, orientation handling, and timing. The table below summarizes the recorded manual test execution.

The test set is not large enough to claim production-level accuracy. It does show that the implemented pipeline works as intended across the main feature paths.

4.2.3 Performance Evaluation

OCR dominates processing time. The checksum and regex detectors run quickly compared with Tesseract. The early-exit design is therefore important: a normal upright image can complete after the first OCR pass, while a difficult image may require several attempts.

For image files, processing time depends on resolution and OCR quality. For PDFs, time increases with page count because each page is converted and processed separately. The current implementation processes PDF pages sequentially. Parallel page processing is a possible future optimization.

Table 4.4: Test Case Execution Summary

Test Case	**Input**	**Expected**	**Actual**	**Status**
TC-01	Clear Aadhaar PNG	One Aadhaar detection	One Aadhaar detection	Pass
TC-02	Single-page PDF	Redaction applied to PDF output	Redaction applied	Pass
TC-03	Multiple PII types	All readable supported types detected	All expected readable types detected	Pass
TC-04	Partial masking	First four characters masked	First four characters masked	Pass
TC-05	Full masking	All characters masked	All characters masked	Pass
TC-06	No-PII document	Zero detections	Zero detections	Pass
TC-07	Invalid Aadhaar	Rejected by Verhoeff validation	Rejected	Pass
TC-08	Names/locations with NER enabled	NER detections where confidence is high	NER detections returned	Pass
TC-09	Word document	HTTP 400 unsupported file type	HTTP 400	Pass
TC-10	Invalid level	HTTP 400 invalid level	HTTP 400	Pass
TC-11	Corrupted image	Processing error response	HTTP 500	Pass
TC-12	90-degree rotated Aadhaar	Detection after orientation handling	Detection returned	Pass
TC-13	180-degree rotated Aadhaar	Detection after orientation handling	Detection returned	Pass
TC-14	Upside-down Aadhaar	Detection after orientation handling	Detection returned	Pass
TC-15	Standard Aadhaar timing	Around or below 300 ms	274–296 ms	Pass
TC-16	Difficult orientation/no-detection case	Under 2000 ms	1582 ms	Pass

NER adds substantial overhead when enabled. The model is preloaded at startup if dependencies are available, which reduces first-request delay, but inference still takes longer than rule-based detection. Users who only need structured identifier redaction can disable NER through the UI or API.

Audit logging adds little overhead compared with OCR. It performs JSON serialization and SHA-256 hashing for one metadata entry per request.

Table 4.5: Qualitative Comparison with Related Redaction Approaches

Capability	**RE-DACT**	**Adobe Acrobat Pro**	**Microsoft Presidio**	**Manual Redaction**
Aadhaar Verhoeff validation	Yes	No built-in Indian-specific flow	Custom recognizer required	Depends on reviewer
PAN structure validation	Yes	No built-in Indian-specific flow	Custom recognizer required	Depends on reviewer
Payment card Luhn validation	Yes	Available through search/manual tools	Yes for credit-card recognizer	Depends on reviewer
Multi-orientation OCR attempts	Yes	Not the focus of the tool	Not the core text analyzer's role	Reviewer dependent
Tamper-evident audit metadata	Basic hash chain	Product logging depends on setup	Not built in as a hash chain	Manual records only
Local web API	Yes	No	Yes as a framework	No
No permanent document storage by default	Yes, short temporary TTL	Local file handling	Depends on integration	Depends on process

The comparison shows where RE-DACT is useful: local processing, Indian structured identifiers, and OCR-to-mask output. It is not a replacement for mature PDF editing suites or a full PII-detection platform. It is a focused prototype for a specific redaction workflow.

The processing time measurements reveal important characteristics about the system's behavior under different input conditions. The measurements were obtained by processing representative documents through the complete pipeline including file upload, OCR, detection, masking, and result preparation.

For correctly oriented documents containing clearly visible Aadhaar numbers, the system achieves processing times in the range of 274 to 296 milliseconds. This fast processing is achieved because the early-exit optimization terminates the multi-orientation pipeline after the first successful detection. The Tesseract OCR with the specified configuration processes the single-page image quickly, and the deterministic validation algorithms complete in negligible time compared to OCR.

The processing time increases significantly when documents require multiple orientation attempts. The worst-case measurement of 1582 milliseconds was observed for a PAN card sample where OCR failed to extract recognizable text in the initial orientations, requiring the system to test multiple rotation and preprocessing combi-

nations before concluding with no detections. This represents approximately a fivefold increase over the best-case scenario and demonstrates the trade-off between comprehensive scanning and processing speed.

The multi-page PDF processing time scales approximately linearly with the number of pages. Each page must be converted to an image, processed through the OCR pipeline, and handled according to detection results. For a three-page PDF with one page containing a detectable Aadhaar number, the total processing time was approximately 900 milliseconds, or about 300 milliseconds per page.

Several deployment considerations affect the practical use of the RE-DACT system in different environments. For local development and testing, the system requires Python 3.11 or compatible version, Tesseract OCR version 5.x or later, and Poppler utilities for PDF conversion. The Python dependencies can be installed via pip from the requirements file.

For containerized deployment, the system can be packaged using Docker. A Dockerfile would include the base Python image, system dependencies for Tesseract and Poppler, Python package installation, and application startup configuration. Containerization simplifies deployment across different environments and enables scaling through orchestration platforms like Kubernetes.

The implementation follows the design described in Chapter 3. The codebase is organized into small Python modules and a static browser interface. The main files include `app.py` for FastAPI application, file validation, endpoint handling, result_TTL_storage, and static-file serving; `redaction.py` for OCR, rotation handling, masking, PDF conversion, and output assembly; `detectors.py` for Aadhaar, PAN, payment card, email, phone, overlap filtering, and masking-range logic; `ner_detector.py` for optional Hugging Face NER pipeline; `audit.py` for JSON-lines audit logging with SHA-256 hash chaining; and `static/` containing HTML, CSS, and JavaScript for the user interface.

The testing methodology follows a systematic approach that verifies both individual component behavior and end-to-end pipeline functionality. Each test case is designed to exercise specific aspects of the system, from input validation through to final output generation.

Unit-level testing focuses on individual detection functions in isolation. The Verhoeff validation function is tested with known valid and invalid Aadhaar numbers. The Luhn validation function is tested with known valid and invalid payment card numbers. The PAN pattern matching is tested with various format-compliant and non-compliant strings.

Integration-level testing verifies that detection functions work correctly when called from the main processing pipeline. The test inputs at this level are real document images that have been manually verified to contain specific PII types.

System-level testing verifies the complete processing pipeline including the frontend interface, API endpoints, background processing, and output file generation. This level of testing requires running the full application and is more time-consuming but necessary to verify that all components work together correctly.

The system depends on both Python packages and system tools. Tesseract OCR is required for text recognition. Poppler utilities are required by pdf2image for PDF-to-image conversion. The Python dependencies are listed in requirements.txt.

For local development and testing, the system requires Python 3.11 or compatible version, Tesseract OCR version 5.x or later, and Poppler utilities for PDF conversion. The Python dependencies can be installed via pip from the requirements file. Creating a virtual environment before installation is recommended to avoid conflicts with other Python projects. The transformers and torch packages for NER support are optional and can be omitted if they are not needed for the deployment. When NER is not needed, the system runs faster because it avoids loading the transformer model into memory.

The hardware requirements are modest for moderate use. A modern laptop with a multi-core CPU can process typical single-page documents in under one second. Multi-page PDFs or documents requiring multiple OCR orientations will take longer. The system does not require GPU acceleration for its core functions, which simplifies deployment and reduces infrastructure costs.

The recorded test results show the system works well on clear Aadhaar samples and that OCR quality is the main limiting factor for PAN, payment cards, and NER. The test set includes Aadhaar cards, PAN cards, payment cards, and mixed documents.

Each document type has different characteristics that affect detection accuracy.

For Aadhaar documents, the 12-digit identifier is usually printed in a consistent format with clear spacing. The Verhoeff validation provides strong filtering against false positives from random digit sequences. In tests with clear, upright Aadhaar images, the system achieved 100

For PAN documents, the ten-character alphanumeric format is less standardized in presentation. The fourth-character holder-type validation helps filter false positives, but the overall detection rate is lower than for Aadhaar. OCR errors on similar-looking characters like O and 0, I and 1, B and 8 contribute to detection failures.

For payment card documents, the Luhn validation provides similar benefits to Verhoeff for reducing false positives. However, payment card numbers may appear in various formats with different spacing and grouping, which can affect pattern matching before validation. The current implementation searches for continuous digit sequences, so card numbers with spaces or dashes may not be detected unless they are stripped during preprocessing.

The processing time measurements reveal important characteristics about the system's behavior under different input conditions. The measurements were obtained by processing representative documents through the complete pipeline including file upload, OCR, detection, masking, and result preparation.

For correctly oriented documents containing clearly visible Aadhaar numbers, the system achieves processing times in the range of 274 to 296 milliseconds. This fast processing is achieved because the early-exit optimization terminates the multi-orientation pipeline after the first successful detection. The Tesseract OCR with the specified configuration processes the single-page image quickly, and the deterministic validation algorithms complete in negligible time compared to OCR.

The processing time increases significantly when documents require multiple orientation attempts. The worst-case measurement of 1582 milliseconds was observed for a PAN card sample where OCR failed to extract recognizable text in the initial orientations, requiring the system to test multiple rotation and preprocessing combinations before concluding with no detections. This represents approximately a fivefold increase over the best-case scenario and demonstrates the trade-off between compre-

hensive scanning and processing speed.

The multi-page PDF processing time scales approximately linearly with the number of pages. Each page must be converted to an image, processed through the OCR pipeline, and handled according to detection results.

Interface behavior verification confirms that the frontend correctly displays processing results. The interface shows total detections, pages processed, processing time in milliseconds, selected redaction level, and file type. The detection breakdown shows counts by type. The detections table displays the detected entity type, masked value, and page number for each detection.

When no PII is found, the API returns a successful response with `download_url` set to null and a message indicating that no PII was detected. The interface treats this as a clean result rather than an error, providing clear feedback to users without unnecessary alarm.

This result structure enables the interface to display comprehensive processing feedback. Users can see what was detected, how long processing took, and download the redacted file when detections were found. The clean-result case provides closure without implying failure.

OCR dominates processing time. The checksum and regex detectors run quickly compared with Tesseract. The early-exit design is therefore important: a normal upright image can complete after the first OCR pass, while a difficult image may require several attempts.

For image files, processing time depends on resolution and OCR quality. For PDFs, time increases with page count because each page is converted and processed separately. The current implementation processes PDF pages sequentially. Parallel page processing is a possible future optimization.

NER adds substantial overhead when enabled. The model is preloaded at startup if dependencies are available, which reduces first-request delay, but inference still takes longer than rule-based detection.

4.3 Summary

The implementation translates the design into a working FastAPI application with a browser interface. The recorded tests show strong Aadhaar performance on clear inputs, more variable PAN/payment-card performance when OCR is challenged, and optional NER behavior that depends on text quality and model fit.

The main engineering strengths are the simple module boundaries, validation-based structured detection, multi-orientation OCR, and short-lived result storage. The main limitations are OCR sensitivity, limited input formats, lack of batch infrastructure, and audit-chain continuity across restarts. The next chapter summarizes the project and outlines future work.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

RE-DACT implements a working prototype for automated redaction of sensitive information in PDF and image documents. The project focuses on Indian identity-document workflows, especially Aadhaar, PAN, and payment card detection, while also supporting email, phone, and optional NER-based name and location detection.

The strongest part of the system is structured validation. Aadhaar candidates are checked with the Verhoeff algorithm, and payment card candidates are checked with the Luhn algorithm. PAN candidates are checked against the official ten-character structure and valid fourth-character holder-type codes. These checks make the detector more reliable than plain pattern matching for the supported identifier types.

The multi-orientation OCR pipeline addresses a practical problem in scanned-document handling. By trying four rotations and two preprocessing states, the system can process documents that are not upright. The early-exit strategy keeps normal cases faster because the pipeline stops as soon as a variant produces a detection.

The masking implementation works at character level using Tesseract bounding boxes. This lets the system apply standard, partial, and full masking without obscuring more text than necessary. The current behavior is: standard masks all but the last four characters, partial masks the first four characters only, and full masks the whole detected value.

The FastAPI backend and plain browser interface make the prototype easy to run and test. The API returns detection statistics, page counts, masked values, processing time, and a temporary download link. The frontend presents the same information through a simple upload-and-results workflow.

The audit log adds basic tamper evidence by linking JSON-lines entries with SHA-256 hashes. This gives the project a foundation for compliance-style record keeping. The current implementation is process-continuous, so a production version should restore the previous hash from disk after restart if it needs one uninterrupted chain.

The evaluation shows that the system works well on clear Aadhaar samples and that OCR quality is the main limiting factor for PAN, payment cards, and NER. The results are useful for validating the prototype, but the test set is too small to claim broad production accuracy. Larger and more varied datasets would be needed before deploying the system in a high-risk environment.

Overall, the project demonstrates that a practical redaction pipeline can be built from OCR, deterministic validation, image masking, web APIs, and short-retention file handling. It solves a focused problem and also makes the remaining limitations visible, which is valuable for future development.

5.2 Future Scope

The first improvement should be better normalization before detection. Aadhaar and payment card numbers may appear with spaces or dashes, and PAN text may be affected by OCR case errors. A normalization layer that preserves coordinate mapping would improve recall without giving up validation.

NER support should also be strengthened. The current model detects general English names and locations, but it is not trained for Indian identity documents or noisy OCR output. Future work could evaluate Indian or multilingual NER models, add address-specific extraction, and use confidence calibration based on document type.

Multilingual OCR is another important extension. Many Indian documents contain Hindi, Tamil, Telugu, Kannada, Bengali, Malayalam, Marathi, Gujarati, Punjabi, or other scripts. Adding Tesseract language packs and script-aware preprocessing would make the system useful beyond English alphanumeric fields.

The audit log can be made stronger. A production version should load the last hash at startup, include a verification command, protect log file permissions, and optionally export signed audit summaries. This would make tamper evidence easier to demonstrate.

Deployment also needs more work. Containerization with Docker would simplify setup. A background task queue such as Celery or RQ could handle larger files and batch jobs. Multiple workers would improve throughput, but shared audit logging and temporary-file cleanup would need careful design.

Security hardening should include file-size limits, rate limiting, authentication, stricter CORS settings, malware scanning where appropriate, and clearer retention controls. Temporary files should be cleaned even after unexpected failures, and production deployments should avoid exposing debug details in error messages.

Performance can be improved through parallel page processing, OCR configuration tuning, and selective orientation attempts based on image metadata. NER can be disabled by default for workflows that only require structured identifiers, keeping the common path faster.

The practical value of the system lies in its combination of features that address specific gaps in the available tooling. The built-in validation for Indian identifier types eliminates the need for custom rule development that existing tools would require. The multi-orientation handling addresses a common failure mode that would otherwise require manual pre-processing. The audit logging provides compliance evidence that most open-source tools lack. The zero-data-retention design addresses data privacy concerns that cloud-based alternatives raise.

The project also demonstrates that effective document processing systems can be built from established open-source components without requiring proprietary software or specialized infrastructure. This accessibility enables adoption by organizations that cannot afford commercial licensing or that have policies against sending sensitive documents to external services.

The path from prototype to production deployment involves addressing several practical considerations beyond the core functionality. These include scaling for higher document volumes, integration with existing document management systems, and operational monitoring for reliability assurance.

Scaling considerations become relevant when the system handles more than occasional document processing. The current single-process design works well for demonstration and moderate use but would benefit from horizontal scaling through container

orchestration for larger deployments. Multiple instances can process documents in parallel, improving throughput for batch operations while maintaining the zero-data-retention principle through stateless request handling.

System integration with existing workflows often requires connecting RE-DACT to document management platforms, email systems, or custom applications. The REST API design facilitates such integration through standard HTTP interfaces. Custom connectors could be developed for specific platforms based on the same API patterns. Webhook notifications could alert downstream systems when processing completes.

Operational monitoring ensures reliable performance over time. Health check endpoints enable integration with orchestration systems for liveness and readiness verification. Metrics on processing times, detection counts, and error rates support capacity planning and performance tuning. Log aggregation enables troubleshooting when issues occur. Alerting on unusual error patterns helps maintain service reliability. The combination of these monitoring capabilities supports operational excellence in production environments.

Beyond the technical improvements identified earlier, the system could benefit from broader community engagement and collaborative development. Open-sourcing the core detection logic would allow security researchers to audit the validation implementations and identify potential false negative scenarios that may not be apparent from limited testing. Public contributions could expand the detection patterns for additional document types and identifier formats relevant to Indian contexts.

The user interface design could also evolve based on user feedback. The current implementation provides basic functionality for uploading documents, selecting redaction levels, and downloading results. Future interface improvements could include batch upload support for processing multiple documents in a single session, progress indicators for long-running operations, configuration options for frequently-used settings, and detailed export options for audit reports.

Training materials and documentation would support broader adoption. Installation guides for different operating systems, configuration options for enterprise deployment, API documentation for integration developers, and troubleshooting guides for common issues would lower the barrier to entry for new users.

Beyond the technical improvements identified earlier, the system could benefit from broader community engagement and collaborative development. Open-sourcing the core detection logic would allow security researchers to audit the validation implementations and identify potential false negative scenarios that may not be apparent from limited testing. Public contributions could expand the detection patterns for additional document types and identifier formats relevant to Indian contexts.

The user interface design could also evolve based on user feedback. The current implementation provides basic functionality for uploading documents, selecting redaction levels, and downloading results. Future interface improvements could include batch upload support for processing multiple documents in a single session, progress indicators for long-running operations, and detailed export options for audit reports.

Training materials and documentation would support broader adoption. Installation guides for different operating systems, configuration options for enterprise deployment, API documentation for integration developers, and troubleshooting guides for common issues would lower the barrier to entry for new users.

These improvements would move RE-DACT from a focused academic prototype toward a more dependable document-redaction tool for Indian identity-data workflows.

The report has documented the complete development journey from problem identification through design, implementation, and evaluation. The system achieves its core objectives while making remaining limitations explicit for future development. Organizations can adopt the current implementation for moderate-volume processing while planning the identified enhancements for broader deployment.

The practical value of the system lies in its combination of features that address specific gaps in the available tooling. The built-in validation for Indian identifier types eliminates the need for custom rule development that existing tools would require. The multi-orientation handling addresses a common failure mode that would otherwise require manual pre-processing. The audit logging provides compliance evidence that most open-source tools lack. The zero-data-retention design addresses data privacy concerns that cloud-based alternatives raise.

The project also demonstrates that effective document processing systems can be built from established open-source components without requiring proprietary software

or specialized infrastructure. This accessibility enables adoption by organizations that cannot afford commercial licensing or that have policies against sending sensitive documents to external services.

Scaling considerations become relevant when the system handles more than occasional document processing. The current single-process design works well for demonstration and moderate use but would benefit from horizontal scaling through container orchestration for larger deployments. Multiple instances can process documents in parallel, improving throughput for batch operations while maintaining the zero-data-retention principle through stateless request handling.

System integration with existing workflows often requires connecting RE-DACT to document management platforms, email systems, or custom applications. The REST API design facilitates such integration through standard HTTP interfaces. Custom connectors could be developed for specific platforms based on the same API patterns. Webhook notifications could alert downstream systems when processing completes.

Operational monitoring ensures reliable performance over time. Health check endpoints enable integration with orchestration systems for liveness and readiness verification. Metrics on processing times, detection counts, and error rates support capacity planning and performance tuning.

Beyond the technical improvements identified earlier, the system could benefit from broader community engagement and collaborative development. Open-sourcing the core detection logic would allow security researchers to audit the validation implementations and identify potential false negative scenarios that may not be apparent from limited testing. Public contributions could expand the detection patterns for additional document types and identifier formats relevant to Indian contexts.

The user interface design could also evolve based on user feedback. The current implementation provides basic functionality for uploading documents, selecting redaction levels, and downloading results. Future interface improvements could include batch upload support for processing multiple documents in a single session, progress indicators for long-running operations, configuration options for frequently-used settings, and detailed export options for audit reports.

Training materials and documentation would support broader adoption. Installa-

tion guides for different operating systems, configuration options for enterprise deployment, API documentation for integration developers, and troubleshooting guides for common issues would lower the barrier to entry for new users.

The compliance landscape continues to evolve as new regulations are enacted and existing frameworks are interpreted more strictly. The system should be designed to adapt to changing requirements without major architectural changes. Modular detection logic allows adding new PII types as regulations define them. Configurable masking strategies let organizations adjust redaction approaches based on their specific compliance requirements. The audit logging provides evidence that can be demonstrated to regulators during compliance reviews.

Research directions beyond the immediate product improvements include exploring privacy-preserving machine learning techniques that could potentially improve detection accuracy without requiring access to sensitive training data. Federated learning approaches could enable model improvements across multiple deployments without centralizing sensitive documents. Configurable masking strategies let organizations adjust redaction approaches based on their specific compliance requirements. The audit logging provides evidence that can be demonstrated to regulators during compliance reviews.

Research directions beyond the immediate product improvements include exploring privacy-preserving machine learning techniques that could potentially improve detection accuracy without requiring access to sensitive training data. Federated learning approaches could enable model improvements across multiple deployments without centralizing sensitive documents. Differential privacy techniques could provide mathematical guarantees about information leakage from detection outputs.

The computational requirements for OCR and validation are modest compared to many machine learning applications, making the system accessible to organizations with limited technical infrastructure. However, further optimization could enable deployment on even more constrained environments such as mobile devices or edge computing nodes. This could enable on-device processing that never transmits sensitive documents over networks, providing stronger privacy guarantees than server-based processing.

The report has documented the complete development journey from problem identification through design, implementation, and evaluation. The system achieves its core objectives while making remaining limitations explicit for future development.

These considerations outline both the immediate technical roadmap and the broader ecosystem development that could support long-term success of the redaction system. The foundation established by the current implementation provides a solid basis for addressing these opportunities.

The core achievements of this project demonstrate the feasibility of building practical document redaction systems for Indian identity documents. The system successfully combines OCR with deterministic validation to achieve reliable detection of structured identifiers. The multi-orientation handling addresses a common real-world problem that would cause simpler systems to fail. The zero-data-retention design provides privacy guarantees that cloud-based alternatives cannot match.

The validation algorithms represent the strongest technical contribution. The Verhoeff implementation for Aadhaar and the Luhn implementation for payment cards provide mathematical certainty about candidate validity that regex-based systems cannot achieve. The PAN holder-type validation adds semantic filtering beyond simple structural matching. These algorithms are efficient enough to run on every candidate without significant performance impact.

The processing pipeline handles the practical challenges of real documents. Rotation, skew, noise, and varying quality all affect OCR accuracy. The eight-attempt orientation strategy ensures that the system finds text regardless of document orientation, while the early-exit optimization keeps fast cases quick. This balance between thoroughness and performance is essential for practical deployment.

The limitations of the current implementation provide direction for future work. OCR quality remains the primary bottleneck for challenging documents. The system does not handle documents with multiple pages efficiently because each page is processed sequentially. The audit logging maintains continuity only within a single process run. These limitations are understood and documented, which enables organizations to make informed decisions about deployment.

The project demonstrates that effective document processing systems can be built from established open-source components without requiring proprietary software or specialized infrastructure. This accessibility enables adoption by organizations that cannot afford commercial licensing or that have policies against sending sensitive documents to external services. The modular architecture allows individual components to be upgraded as technology improves.

The path from prototype to production deployment involves addressing several practical considerations beyond the core functionality. These include scaling for higher document volumes, integration with existing document management systems, and operational monitoring for reliability assurance.

Scaling considerations become relevant when the system handles more than occasional document processing. The current single-process design works well for demonstration and moderate use but would benefit from horizontal scaling through container orchestration for larger deployments.

References

1. R. Smith, "An Overview of the Tesseract OCR Engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

2. Tesseract OCR, "Tesseract User Manual," 2026. [Online]. Available: https://tesseract-ocr.github.io/tessdoc/

3. J. Verhoeff, *Error Detecting Decimal Codes*. Mathematical Centre Tracts, no. 29. Amsterdam: Mathematisch Centrum, 1969.

4. H. P. Luhn, "Computer for Verifying Numbers," U.S. Patent 2,950,048, Aug. 23, 1960.

5. Income Tax Department, Government of India, "How PAN is formed and how it gets its unique identity?" [Online]. Available: https://www.incometaxindia.gov.in/w/how-pan-is-formed-and-how-it-gets-its-unique-identity-

6. Government of India, "The Digital Personal Data Protection Act, 2023," India Code. [Online]. Available: https://www.indiacode.nic.in/bitstream/123456789/22037/1/a20 22.pdf

7. Data Protection Commission, Ireland, "Data Protection Commission announces conclusion of inquiry into Meta Ireland," May 22, 2023. [Online]. Available: https://www.dataprotection.ie/en/news-media/press-releases/Data-Protection-Commission announces-conclusion-of-inquiry-into-Meta-Ireland

8. Adobe, "Redact sensitive content in Acrobat Pro," 2025. [Online]. Available: https://helpx.adobe.com/acrobat/using/removing-sensitive-content-pdfs.html

9. Microsoft, "PII entities supported by Presidio." [Online]. Available: https://microsoft.github

10. Hugging Face, "dslim/bert-base-NER model card." [Online]. Available: https://huggingface. base-NER

11. D. Sánchez and M. Batet, "C-sanitized: a privacy model for document redaction and sanitization," *Journal of the Association for Information Science and Technology*, vol. 67, no. 1, pp. 148–163, 2016.

12. C. A. Lee and K. Woods, "Automated Redaction of Private and Personal Data in Collections: Toward Responsible Stewardship of Digital Heritage," in *Proceedings of The Memory of the World in the Digital Age: Digitization and Preservation*, Vancouver, BC, Canada, 2012, pp. 298–312.

13. T. Orekondy, M. Fritz, and B. Schiele, "Connecting Pixels to Privacy and Utility: Automatic Redaction of Private Information in Images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8466–8475.

14. S. Peng, M.-J. Huang, M. Wu, and J. Wei, "Transforming Redaction: How AI is Revolutionizing Data Protection," *arXiv preprint arXiv:2409.15308*, 2024.

15. L. Garza, A. Kotal, A. Piplai, L. Elluri, P. K. Das, and A. Chadha, "PRvL: Quantifying the Capabilities and Risks of Large Language Models for PII Redaction," *arXiv preprint arXiv:2508.05545*, 2025.

16. P. Thetbanthad, B. Sathanarugsawait, and P. Praneetpolgrang, "Automated Redaction of Personally Identifiable Information on Drug Labels Using Optical Character Recognition and Large Language Models for Compliance with Thailand's Personal Data Protection Act," *Applied Sciences*, vol. 15, no. 9, p. 4923, 2025.

17. S. Ramírez, "FastAPI: Modern, Fast Web Framework for Building APIs with Python." [Online]. Available: https://fastapi.tiangolo.com

18. OpenCV, "Open Source Computer Vision Library." [Online]. Available: https://opencv.org

19. python-pdf2image, "pdf2image documentation." [Online]. Available: https://pdf2image.read

APPENDIX A

LIST OF PAPERS

A.1 List of Published Papers

A.2 List of Accepted Papers

A.3 List of Communicated Papers

APPENDIX B

PLAGIARISM REPORT

B.1 Plagiarism Report

B.2 AI Content Report